```java
package com.tup.tfi.control107;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.
SpringBootApplication;

@SpringBootApplication
public class App {

    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }

}

```

```
1 # Placeholder
2
```

```java
package com.tup.tfi.control107.dto;

import java.time.LocalDateTime;
import java.util.Collections;
import java.util.List;

public record ErrorDTO(
        LocalDateTime timestamp,
        int status,
        String error,
        List<String> detalles
) {
    public static ErrorDTO of(int status, String error, List<String> detalles) {
        return new ErrorDTO(LocalDateTime.now(), status, error, detalles);
    }
    public static ErrorDTO simple(int status, String error, String mensaje) {
        return new ErrorDTO(LocalDateTime.now(), status, error, Collections.singletonList(mensaje));
    }
}

```

```java
package com.tup.tfi.control107.dto;

public record LoginDTO(
        String access_token,
        Long expires_in,
        String token_type,
        String scope
) {
}
```

```java
package com.tup.tfi.control107.dto;

import com.tup.tfi.control107.model.entity.*;
import com.tup.tfi.control107.model.enums.EstadoTurno
;
import com.tup.tfi.control107.model.enums.
HorarioTurno;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDate;
import java.util.Set;
import java.util.stream.Collectors;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class TurnoDTO {
    private Long id;
    private LocalDate fecha;
    private HorarioTurno horario;
    private EstadoTurno estado;
    private UsuarioDTO admin;
    private UsuarioDTO enfermero;
    private String medico;
    private String chofer;
    private AmbulanciaDTO ambulancia;
    private String observacionesIngreso;
    private String observacionesEgreso;
    private Set<ControlInsumoDTO> controlInsumos;

    public TurnoDTO(Turno t) {
        if (null == t) return;
        this.id = t.getId();
        this.fecha = t.getFecha();
        this.horario = t.getHorario();
        this.estado = t.getEstado();
        this.chofer = t.getNombreChofer();
```

```
40          this.medico = t.getNombreMedico();
41          this.observacionesIngreso = t.
    getObservacionesIngreso();
42          this.observacionesEgreso = t.
    getObservacionesEgreso();
43          if (t.getEnfermero() != null) {
44              this.enfermero = new UsuarioDTO(t.
    getEnfermero());
45          }
46          if (t.getAdmin() != null) {
47              this.admin = new UsuarioDTO(t.getAdmin
    ());
48          }
49          if (t.getAmbulancia() != null) {
50              this.ambulancia = new AmbulanciaDTO(t.
    getAmbulancia());
51          }
52          if (t.getControlInsumos() != null) {
53              this.controlInsumos = t.getControlInsumos
    ().stream()
54                      .map(ControlInsumoDTO::new)
55                      .collect(Collectors.toSet());
56          }
57      }
58
59      @Data
60      @Builder
61      @NoArgsConstructor
62      @AllArgsConstructor
63      public static class ControlInsumoDTO {
64          private String nombre;
65          private String categoria;
66          private boolean esCritico;
67          private int cantidadIngreso;
68          private Integer cantidadEgreso;
69          private int ingresoFaltante;
70          private Integer cantidadUsada;
71          private Integer cantidadRestockeo;
72
73          public ControlInsumoDTO(ControlInsumo
    controlInsumo) {
```

```
 74              if (null == controlInsumo) return;
 75              this.nombre = controlInsumo.getInsumo().
    getNombre();
 76              this.categoria = controlInsumo.getInsumo
    ().getCategoria().getNombre();
 77              this.esCritico = controlInsumo.getInsumo
    ().isCritico();
 78              this.cantidadIngreso = controlInsumo.
    getCantidadIngreso();
 79              this.cantidadEgreso = controlInsumo.
    getCantidadEgreso();
 80              this.ingresoFaltante = controlInsumo.
    getIngresoFaltante();
 81              this.cantidadUsada = controlInsumo.
    getCantidadUsada();
 82              this.cantidadRestockeo = controlInsumo.
    getCantidadRestockeo();
 83          }
 84      }
 85
 86      @Data
 87      @Builder
 88      @NoArgsConstructor
 89      @AllArgsConstructor
 90      public static class UsuarioDTO {
 91          private Long id;
 92          private String email;
 93
 94          public UsuarioDTO(Usuario u) {
 95              if (null == u) return;
 96              this.id = u.getId();
 97              this.email = u.getEmail();
 98          }
 99      }
100
101      @Data
102      @Builder
103      @NoArgsConstructor
104      @AllArgsConstructor
105      public static class AmbulanciaDTO {
106          private Long id;
```

```
107         private String patente;
108
109         public AmbulanciaDTO(Ambulancia a) {
110             if (null == a) return;
111             this.id = a.getId();
112             this.patente = a.getPatente();
113         }
114     }
115
116 }
117
```

```java
package com.tup.tfi.control107.dto;

import com.tup.tfi.control107.model.entity.Insumo;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class InsumoDTO {
    private Long id;
    private String nombre;
    private String categoria;
    private int puntoControl;
    private boolean esCritico;

    public InsumoDTO(Insumo i) {
        if (null == i) return;
        this.id = i.getId();
        this.nombre = i.getNombre();
        this.puntoControl = i.getPuntoControl();
        this.esCritico = i.isCritico();
        if (i.getCategoria() != null) {
            this.categoria = i.getCategoria().
  getNombre();
        } else {
            this.categoria = "SIN CATEGORIA";
        }
    }
}

```

```java
package com.tup.tfi.control107.dto;

import com.tup.tfi.control107.model.entity.Usuario;
import com.tup.tfi.control107.model.enums.RolUsuario;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class UsuarioDTO {
    private Long id;
    private String nombre;
    private String dni;
    private String email;
    private RolUsuario rol;
    private boolean activo;

    public UsuarioDTO(Usuario u) {
        if (null == u) return;
        this.id = u.getId();
        this.nombre = u.getNombre();
        this.dni = u.getDni();
        this.email = u.getEmail();
        this.rol = u.getRol();
        this.activo = u.isActivo();
    }
}

```

```java
package com.tup.tfi.control107.dto;

import com.tup.tfi.control107.model.entity.Ambulancia
;
import lombok.*;

import java.util.Set;
import java.util.stream.Collectors;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class AmbulanciaDTO {
    private Long id;
    private String patente;
    private boolean disponible;
    private Set<InsumoDTO> insumos;

    public AmbulanciaDTO(Ambulancia a) {
        if (null == a) return;
        this.id = a.getId();
        this.patente = a.getPatente();
        this.disponible = a.isDisponible();
        if (a.getInsumos() != null) {
            this.insumos = a.getInsumos().stream()
                    .map(InsumoDTO::new)
                    .collect(Collectors.toSet());
        } else {
            this.insumos = Set.of();
        }
    }
}

```

```java
package com.tup.tfi.control107.dto;

import com.tup.tfi.control107.model.entity.
CategoriaInsumo;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class CategoriaInsumoDTO {
    private Long id;
    private String nombre;
    private String color;

    public CategoriaInsumoDTO(CategoriaInsumo c) {
        if (null == c) return;
        this.id = c.getId();
        this.nombre = c.getNombre();
        this.color = c.getColor();
    }
}

```

```java
package com.tup.tfi.control107.dto;

import lombok.*;

import java.util.List;

@Getter
@Builder
public class ResumenOperativoDTO {
    private long turnosActivos;
    private long ambulanciasDisponibles;
    private long ambulanciasTotales;
    private long alertas;
    private List<TurnoDTO> turnosConInconsistencias;
    private List<AmbulanciaDTO> listaAmbulancias;

}

```

```java
package com.tup.tfi.control107.dto.request;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record LoginRequestDTO (
        @NotBlank(message = "El mail es obligatorio")
        @Email(message = "Formato de email inválido")
        String mail,

        @NotBlank(message = "La clave es obligatoria")
        @Size(min = 6, message = "La clave debe tener al menos 6 caracteres")
        String clave
){
}

```

```java
package com.tup.tfi.control107.dto.request;

import com.tup.tfi.control107.model.entity.Ambulancia;
import com.tup.tfi.control107.model.entity.Turno;
import com.tup.tfi.control107.model.entity.Usuario;
import com.tup.tfi.control107.model.enums.EstadoTurno;
import com.tup.tfi.control107.model.enums.HorarioTurno;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class TurnoRequestDTO {
    private LocalDate fecha;
    private HorarioTurno horario;
    private Long adminId;
    private Long enfermeroId;
    private Long ambulanciaId;
    private String nombreChofer;
    private String nombreMedico;

    public Turno toTurno(@NotNull Usuario admin, @NotNull Usuario enfermero, @NotNull Ambulancia ambulancia) {
        Turno turno = Turno.builder()
                .fecha(fecha)
                .horario(horario)
                .nombreChofer(nombreChofer)
                .admin(admin)
                .enfermero(enfermero)
                .nombreChofer(nombreChofer)
                .nombreMedico(nombreMedico)
```

```
37                  .estado(EstadoTurno.PROGRAMADO)
38                  .build();
39
40          turno.setAmbulancia(ambulancia);
41          return turno;
42      }
43
44      public void aplicar(Turno t, Ambulancia
    ambulancia, Usuario enfermero) {
45          if (null != ambulancia) {
46              t.setAmbulancia(ambulancia);
47          }
48          if (null != enfermero) {
49              t.setEnfermero(enfermero);
50          }
51          if (null != fecha) {
52              t.setFecha(fecha);
53          }
54          if (null != horario) {
55              t.setHorario(horario);
56          }
57          if (null != nombreChofer) {
58              t.setNombreChofer(nombreChofer);
59          }
60          if (null != nombreMedico) {
61              t.setNombreMedico(nombreMedico);
62          }
63          t.setUpdatedAt(LocalDateTime.now());
64      }
65  }
66
```

```java
package com.tup.tfi.control107.dto.request;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class CancelarTurnoDTO {
    private String observaciones;
}

```

```java
package com.tup.tfi.control107.dto.request;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.util.List;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class IngresoEgresoDTO {
    private Long turnoId;
    private List<ControlInsumoRequestDTO> insumos;
    private String observaciones;
}

```

```java
package com.tup.tfi.control107.dto.request;

import com.tup.tfi.control107.model.entity.
CategoriaInsumo;
import com.tup.tfi.control107.model.entity.Insumo;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class InsumoRequestDTO {
    private String nombre;
    private Long categoriaId;
    private Integer puntoControl;
    private boolean critico;

    public Insumo toInsumo(@NotNull CategoriaInsumo
categoria) {
        return new Insumo(nombre, categoria,
puntoControl, critico);
    }

    public void aplicar(Insumo i, CategoriaInsumo
categoria) {
        if (null != categoria) {
            i.setCategoria(categoria);
        }
        if (null != nombre) {
            i.setNombre(nombre);
        }
        if (null != puntoControl) {
            i.setPuntoControl(puntoControl);
        }
        i.setCritico(critico);
        i.setUpdatedAt(LocalDateTime.now());
    }
```

```
38 }
39
```

```java
package com.tup.tfi.control107.dto.request;

import com.tup.tfi.control107.model.entity.Usuario;
import com.tup.tfi.control107.model.enums.RolUsuario;
import jakarta.validation.constraints.Email;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class UsuarioRequestDTO {
    private String nombre;
    private String dni;
    @Email(message = "Formato de mail inválido")
    private String email;
    private RolUsuario rol;
    private String clave;
    private boolean activo = true;

    public Usuario toUsuario(Usuario.Clave clave) {
        return Usuario.builder()
                .nombre(nombre)
                .dni(dni)
                .email(email.toLowerCase().trim())
                .rol(rol)
                .clave(clave)
                .activo(activo)
                .build();
    }

    public void aplicar(Usuario u) {
        if (null != nombre) {
            u.setNombre(nombre);
        }
        if (null != dni) {
            u.setDni(dni);
        }
```

```
42          if (null != email) {
43              u.setEmail(email.toLowerCase().trim());
44          }
45          if (null != rol) {
46              u.setRol(rol);
47          }
48          u.setActivo(activo);
49          u.setUpdatedAt(LocalDateTime.now());
50      }
51 }
52
```

```java
package com.tup.tfi.control107.dto.request;

import com.tup.tfi.control107.model.entity.Ambulancia;
import com.tup.tfi.control107.model.entity.Insumo;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;
import java.util.Set;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class AmbulanciaRequestDTO {
    private String patente;
    private boolean disponible = true;
    private Set<Long> insumos = Set.of();

    public Ambulancia toAmbulancia(@NotNull Set<Insumo> insumos) {
        return new Ambulancia(patente.toUpperCase(), disponible, insumos);
    }

    public void aplicar(Ambulancia ambulancia, Set<Insumo> insumos) {
        if (null != patente) {
            ambulancia.setPatente(patente.toUpperCase());
        }
        ambulancia.setDisponible(disponible);
        if (null != insumos) {
            ambulancia.setInsumos(insumos);
        }
        ambulancia.setUpdatedAt(LocalDateTime.now());
    }
}

```

```java
package com.tup.tfi.control107.dto.request;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class ControlInsumoRequestDTO {
    private Long id;
    private int cantidad;
    private Integer cantidadRestockeo;
}

```

```java
package com.tup.tfi.control107.dto.request;

import com.tup.tfi.control107.model.entity.CategoriaInsumo;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.apache.commons.lang3.StringUtils;

import java.time.LocalDateTime;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class CategoriaInsumoRequestDTO {
    private String nombre;
    private String color;

    public CategoriaInsumo toCategoriaInsumo() {
        return CategoriaInsumo.builder()
                .nombre(StringUtils.stripAccents(nombre.toUpperCase()))
                .color(color.toUpperCase())
                .build();
    }

    public void aplicar(CategoriaInsumo c) {
        if (null != nombre) {
            c.setNombre(StringUtils.stripAccents(nombre.toUpperCase()));
        }
        if (null != color) {
            c.setColor(color.toUpperCase());
        }
        c.setUpdatedAt(LocalDateTime.now());
    }
}

```

```java
package com.tup.tfi.control107.util;

import com.tup.tfi.control107.dto.UsuarioDTO;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import lombok.Getter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;

@Component
public class JwtUtil {

    @Value("${security.jwt.secret-key: zP8u6fW7vB4n9m2K5jH3gD1sA6dQ9wE0rT7yU4iO1pL3kJ5hG8fD2sS4aA0zX9cC}")
    private String secretKey;

    @Getter
    @Value("${security.jwt.expiration-time:3600000}")
    private long expirationTime;

    public String createToken(UsuarioDTO usuario) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("rol", usuario.getRol().name());
        claims.put("mail", usuario.getEmail());

        return Jwts.builder()
                .subject(usuario.getId().toString())
                .claims(claims)
                .issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + expirationTime))
                .signWith(getSecretKey())
```

```
38                  .compact();
39      }
40
41      public Claims extractClaims(String token) {
42          return Jwts.parser()
43                  .verifyWith(getSecretKey())
44                  .build()
45                  .parseSignedClaims(token)
46                  .getPayload();
47      }
48
49      public boolean isTokenExpired(String token) {
50          Claims claims = extractClaims(token);
51          Date expiration = claims.getExpiration();
52          return expiration.before(new Date(System.
    currentTimeMillis()));
53      }
54
55
56      private SecretKey getSecretKey() {
57          if (null == secretKey || secretKey.isEmpty
    ()) {
58              return Jwts.SIG.HS256.key().build();
59          } else {
60              return Keys.hmacShaKeyFor(secretKey.
    getBytes(StandardCharsets.UTF_8));
61          }
62      }
63 }
64
```

```java
package com.tup.tfi.control107.util;

import com.tup.tfi.control107.model.entity.Usuario;

import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.security.spec.InvalidKeySpecException;
import java.util.Arrays;
import java.util.Base64;
import java.util.Random;

public class PasswordUtil {
    private static final Random RANDOM = new SecureRandom();
    private static final int ITERATIONS = 10000;
    private static final int KEY_LENGTH = 256;

    /**
     * static utility class
     */
    private PasswordUtil() { }

    /**
     * Returns a random salt to be used to hash a password.
     *
     * @return a 16 bytes random salt
     */
    public static String getNextSalt() {
        byte[] salt = new byte[16];
        RANDOM.nextBytes(salt);
        return Base64.getEncoder().encodeToString(salt);

    }

    public static Usuario.Clave hash(char[] plainPassword, String userSalt) {
        String salt = null != userSalt ? userSalt :
```

```
37 getNextSalt();
38         PBEKeySpec spec = new PBEKeySpec(
   plainPassword, Base64.getDecoder().decode(salt),
   ITERATIONS, KEY_LENGTH);
39         Arrays.fill(plainPassword, Character.
   MIN_VALUE);
40         try {
41             SecretKeyFactory skf = SecretKeyFactory.
   getInstance("PBKDF2WithHmacSHA1");
42             String hash = Base64.getEncoder().
   encodeToString(skf.generateSecret(spec).getEncoded
   ());
43             return new Usuario.Clave(hash, salt);
44         } catch (NoSuchAlgorithmException |
   InvalidKeySpecException e) {
45             throw new AssertionError("Error while
   hashing a password: " + e.getMessage(), e);
46         } finally {
47             spec.clearPassword();
48         }
49     }
50
51     public static boolean verificarClave(char[]
   password, String salt, char[] expectedHash) {
52         char[] pwdHash = hash(password, salt).getHash
   ().toCharArray();
53         Arrays.fill(password, Character.MIN_VALUE);
54         if (pwdHash.length != expectedHash.length)
   return false;
55         for (int i = 0; i < pwdHash.length; i++) {
56             if (pwdHash[i] != expectedHash[i]) return
    false;
57         }
58         return true;
59     }
60
61 }
62
```

```
1 # Placeholder
2
```

```java
package com.tup.tfi.control107.model.enums;

public enum RolUsuario {
    ADMINISTRADOR,
    ENFERMERO
}

```

```java
package com.tup.tfi.control107.model.enums;

public enum EstadoTurno {
    PROGRAMADO,
    EN_CURSO,
    FINALIZADO,
    CANCELADO
}

```

```java
package com.tup.tfi.control107.model.enums;

import lombok.AllArgsConstructor;
import lombok.Getter;

@AllArgsConstructor
@Getter
public enum HorarioTurno {
    MANIANA("06:00","14:00"),
    TARDE("14:00","22:00"),
    NOCHE("22:00","06:00");

    private final String horaDesde;
    private final String horaHasta;
}

```

```java
package com.tup.tfi.control107.model.entity;

import com.tup.tfi.control107.model.enums.EstadoTurno;
import com.tup.tfi.control107.model.enums.HorarioTurno;
import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "turnos")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Turno extends BaseEntity {

    @Column(nullable = false)
    private LocalDate fecha;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private HorarioTurno horario;

    @Builder.Default
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private EstadoTurno estado = EstadoTurno.PROGRAMADO;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "ambulancia_id", nullable = false)
    private Ambulancia ambulancia;

    @ManyToOne
```

```
38      @JoinColumn(name = "enfermero_id")
39      private Usuario enfermero;
40
41      @ManyToOne
42      @JoinColumn(name = "admin_id")
43      private Usuario admin;
44
45      @Column(name = "nombre_chofer", nullable = false)
46      private String nombreChofer;
47
48      @Column(name = "nombre_medico", nullable = false)
49      private String nombreMedico;
50
51      @Column(name = "observaciones_ingreso",
    columnDefinition = "TEXT", length = 1000)
52      private String observacionesIngreso;
53
54      @Column(name = "observaciones_egreso",
    columnDefinition = "TEXT", length = 1000)
55      private String observacionesEgreso;
56
57      /**
58       * Entidad que mapea el tipo de Insumo con el
    control de stock al ingreso,
59       * egreso, faltantes al empezar el turno y
    cantidad usada.
60       */
61      @Builder.Default
62      @OneToMany(mappedBy = "turno", cascade =
    CascadeType.ALL, orphanRemoval = true)
63      private List<ControlInsumo> controlInsumos = new
    ArrayList<>();
64
65
66      /**
67       * Asigna la ambulancia al turno y carga la lista
     base de control de insumos
68       * a partir de los de la ambulancia
69       * @throws IllegalStateException si el turno ya
    tiene una lista de control cargada
70       */
```

```
71      public void setAmbulancia(Ambulancia ambulancia
    ) {
72          if (!controlInsumos.isEmpty()) {
73              throw new IllegalStateException("No se
    puede cambiar la ambulancia de un turno que ya tiene
     control de insumos");
74          }
75          this.ambulancia = ambulancia;
76          ambulancia.getInsumos().stream()
77                  .map(i -> ControlInsumo.builder()
78                          .insumo(i)
79                          .turno(this)
80                          .cantidadIngreso(i.
    getPuntoControl())
81                          .ingresoFaltante(0)
82                          .cantidadRestockeo(0)
83                          .build())
84                  .forEach(controlInsumos::add);
85      }
86  }
87
```

```java
package com.tup.tfi.control107.model.entity;

import jakarta.persistence.*;
import lombok.*;

/**
 * Representa el tipo de insumo utilizado en las ambulancias
 * y la cantidad esperada de stock que debe contener, NO es stock de deposito
 */
@Entity
@Table(name = "insumos")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Insumo extends BaseEntity {

    @Column(nullable = false)
    private String nombre;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "categoria_id", nullable = false)
    private CategoriaInsumo categoria;

    // Cantidad baseline esperada
    @Column(name = "punto_control", nullable = false)
    private int puntoControl;

    @Builder.Default
    @Column(name = "critico", nullable = false)
    private boolean critico = false;
}

```

```java
package com.tup.tfi.control107.model.entity;

import com.tup.tfi.control107.model.enums.RolUsuario;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "usuarios")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Usuario extends BaseEntity {

    @Column(nullable = false)
    private String nombre;

    @Column(unique = true, nullable = false)
    private String dni;

    @Column(unique = true, nullable = false)
    private String email;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private RolUsuario rol;

    @Embedded
    private Clave clave;

    @Builder.Default
    @Column(nullable = false)
    private boolean activo = true;

    @Table(name = "usuario_clave")
    @Getter
    @Setter
    @NoArgsConstructor
    @AllArgsConstructor
    @Builder
```

```
42      @Embeddable
43      public static class Clave {
44
45          @Column(nullable = true)
46          private String hash;
47
48          @Column(nullable = true)
49          private String salt;
50
51      }
52  }
53
```

```java
package com.tup.tfi.control107.model.entity;

import jakarta.persistence.*;
import lombok.*;

import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "ambulancias")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Ambulancia extends BaseEntity {

    @Column(nullable = false, unique = true)
    private String patente;

    @Builder.Default
    @Column(nullable = false)
    private boolean disponible = true;

    /**
     * Insumos que tiene la ambulancia y la cantidad de control que deberia tener siempre
     * para empezar un turno, no es stock de deposito.
     */
    @ManyToMany
    @JoinTable(
            name = "ambulancia_insumo",
            joinColumns = @JoinColumn(name = "ambulancia_id"),
            inverseJoinColumns = @JoinColumn(name = "insumo_id")
    )
    private Set<Insumo> insumos = new HashSet<>();

    public void addInsumo(Insumo insumo) {
```

```
38          this.insumos.add(insumo);
39      }
40
41      public void removeInsumo(Insumo insumo) {
42          this.insumos.remove(insumo);
43      }
44
45  }
46
```

```java
package com.tup.tfi.control107.model.entity;

import jakarta.persistence.*;
import lombok.*;
import lombok.experimental.SuperBuilder;

import java.time.LocalDateTime;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@SuperBuilder
@ToString
@MappedSuperclass
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy= GenerationType.IDENTITY)
    protected Long id;

    @Builder.Default
    @Column(name = "created_at", nullable = false)
    protected final LocalDateTime createdAt = LocalDateTime.now();

    @Builder.Default
    @Column(name = "updated_at", nullable = false)
    protected LocalDateTime updatedAt = LocalDateTime.now();

    @Builder.Default
    @Column(nullable = false)
    protected boolean eliminado = false;

    public void borradoLogico() {
        eliminado = true;
    }
}

```

```java
package com.tup.tfi.control107.model.entity;

import jakarta.persistence.*;
import lombok.*;

/**
 * Registro de control de insumos utilizados en un turno.
 */
@Entity
@Table(name = "control_insumos")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ControlInsumo extends BaseEntity{

    @ManyToOne
    @JoinColumn(name = "turno_id")
    private Turno turno;

    @ManyToOne
    @JoinColumn(name = "insumo_id")
    private Insumo insumo;

    @Column(name = "cantidad_ingreso")
    private int cantidadIngreso;

    @Column(name = "cantidad_egreso")
    private Integer cantidadEgreso;

    @Column(name = "cantidad_restockeo")
    private int cantidadRestockeo;

    @Column(name = "ingreso_faltante")
    private int ingresoFaltante;

    @Column(name = "cantidad_usada")
    private Integer cantidadUsada;

```

```
41      public void informarIngreso(int cantidadIngreso
    ) {
42          this.cantidadIngreso = cantidadIngreso;
43          this.ingresoFaltante = Math.max(0, this.
    insumo.getPuntoControl() - cantidadIngreso);
44      }
45
46      public void informarEgreso(int cantidadEgreso,
    int cantidadRestockeo) {
47          this.cantidadEgreso = cantidadEgreso;
48          this.cantidadRestockeo = cantidadRestockeo;
49          this.cantidadUsada = Math.max(0, this.
    cantidadIngreso + cantidadRestockeo - cantidadEgreso
    );
50      }
51  }
52
```

```java
package com.tup.tfi.control107.model.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Table;
import lombok.*;

@Entity
@Table(name = "categorias")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class CategoriaInsumo extends BaseEntity{

    @Column(nullable = false, unique = true)
    private String nombre;

    // Hexadecimal, example: #FF0000
    @Column(nullable = false)
    private String color;
}

```

```
1 # Placeholder
2
```

```java
package com.tup.tfi.control107.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import org.springframework.web.filter.CorsFilter;

import java.util.Arrays;
import java.util.List;

/**
 * Allow any request from any origin for testing and development purposes.
 */
@Configuration
public class CorsConfig {

    @Value("${app.cors.allowed-origins:*}")
    private String corsAllowedOrigins;

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        CorsConfiguration config = new CorsConfiguration();

        config.setAllowCredentials(true);
        config.setAllowedOriginPatterns(List.of(corsAllowedOrigins.split(",")));
        config.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
```

```
31          config.setAllowedHeaders(List.of("*"));
32          config.setExposedHeaders(Arrays.asList(
33                  "Authorization",
34                  "Content-Type",
35                  "X-Total-Count"
36          ));
37          config.setMaxAge(3600L);
38          source.registerCorsConfiguration("/**",
    config);
39          return source;
40      }
41
42      @Bean
43      public CorsFilter corsFilter() {
44          return new CorsFilter(corsConfigurationSource
    ());
45      }
46  }
```

```java
package com.tup.tfi.control107.config;

import com.tup.tfi.control107.dto.UsuarioDTO;
import com.tup.tfi.control107.dto.request.IngresoEgresoDTO;
import com.tup.tfi.control107.dto.request.ControlInsumoRequestDTO;
import com.tup.tfi.control107.dto.request.TurnoRequestDTO;
import com.tup.tfi.control107.dto.request.UsuarioRequestDTO;
import com.tup.tfi.control107.model.entity.*;
import com.tup.tfi.control107.model.enums.*;
import com.tup.tfi.control107.repository.*;
import com.tup.tfi.control107.service.TurnoService;
import com.tup.tfi.control107.service.UsuarioService;
import lombok.RequiredArgsConstructor;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Component;

import java.time.LocalDate;
import java.util.List;
import java.util.Set;

@Component
@RequiredArgsConstructor
@Profile("dev")
public class DataLoader implements CommandLineRunner {

    private final UsuarioRepository usuarioRepository;
    private final UsuarioService usuarioService;
    private final CategoriaRepository categoriaRepository;
    private final InsumoRepository insumoRepository;
    private final AmbulanciaRepository ambulanciaRepository;
    private final TurnoService turnoService;
```

```
33
34      @Override
35      public void run(String... args) {
36          if (usuarioRepository.count() > 0) {
37              return; // Ya existen datos
38          }
39
40          // 1. Cargar Usuarios
41          UsuarioRequestDTO adminDto = new
    UsuarioRequestDTO(
42                  "Hugo Catalan",
43                  "30111222",
44                  "hugo.catalan@control107.gob.ar",
45                  RolUsuario.ADMINISTRADOR,
46                  "1234",
47                  true
48          );
49
50          UsuarioRequestDTO enfermeroDto = new
    UsuarioRequestDTO(
51                  "Andrés Bonelli",
52                  "32333444",
53                  "andres.bonelli@control107.gob.ar",
54                  RolUsuario.ENFERMERO,
55                  "1234",
56                  true
57          );
58
59          UsuarioDTO admin = usuarioService.
    crearUsuario(adminDto);
60          UsuarioDTO enfermero = usuarioService.
    crearUsuario(enfermeroDto);
61
62          String medico = "Matías Carro";
63          String chofer = "Juan Perez";
64
65          // 2.Cargar Categorias
66          CategoriaInsumo farma = categoriaRepository.
    save(CategoriaInsumo.builder().nombre("FARMACOLOGICOS
    ").color("#D9574C").build());
67          CategoriaInsumo  descartables =
```

```
67 categoriaRepository.save(CategoriaInsumo.builder().
   nombre("DESCARTABLES").color("#087E8B").build());
68         CategoriaInsumo  viaAerea =
   categoriaRepository.save(CategoriaInsumo.builder().
   nombre("VIA AEREA").color("#3978A8").build());
69         CategoriaInsumo  oxigeno =
   categoriaRepository.save(CategoriaInsumo.builder().
   nombre("OXIGENO").color("#E47828").build());
70         CategoriaInsumo  curacion =
   categoriaRepository.save(CategoriaInsumo.builder().
   nombre("CURACION").color("#087E8B").build());
71         CategoriaInsumo  equipamiento =
   categoriaRepository.save(CategoriaInsumo.builder().
   nombre("EQUIPAMIENTO").color("#3978A8").build());
72
73         // 3. Cargar Catálogo de Insumos
74         Insumo adrenalina = insumoRepository.save(
   Insumo.builder().nombre("Adrenalina 1mg").categoria(
   farma).puntoControl(20).critico(false).build());
75         Insumo atropina = insumoRepository.save(
   Insumo.builder().nombre("Atropina 1mg").categoria(
   farma).puntoControl(5).critico(false).build());
76         Insumo gasas = insumoRepository.save(Insumo.
   builder().nombre("Gasas Estériles (Sobres)").
   categoria(curacion).puntoControl(30).critico(false).
   build());
77         Insumo guantes = insumoRepository.save(
   Insumo.builder().nombre("Guantes de Nitrilo M").
   categoria(descartables).puntoControl(100).critico(
   false).build());
78         Insumo tuboO2Portatil = insumoRepository.
   save(Insumo.builder().nombre("Tubo Portátil Presion
   de oxigeno medido en bares").categoria(viaAerea).
   puntoControl(60).critico(true).build());
79         Insumo tuboO2Pesado = insumoRepository.save(
   Insumo.builder().nombre("Tubo grande Presion de
   oxigeno, medido en Bares").categoria(oxigeno).
   puntoControl(60).critico(true).build());
80
81         var insumos = List.of(adrenalina, atropina,
   gasas, guantes, tuboO2Portatil, tuboO2Pesado);
```

```
 82          var insumosSet = Set.of(adrenalina, atropina
    , gasas, guantes, tuboO2Portatil, tuboO2Pesado);
 83
 84          // 4. Cargar Ambulancias
 85          Ambulancia m01 = crearAmbulanciaConInsumos("
    AE 107 AB", insumosSet);
 86          Ambulancia m02 = crearAmbulanciaConInsumos("
    AE 107 CD", insumosSet);
 87          Ambulancia m03 = crearAmbulanciaConInsumos("
    AE 107 EF", insumosSet);
 88          Ambulancia m04 = crearAmbulanciaConInsumos("
    AE 107 GH", insumosSet);
 89
 90
 91          // 5. Crear Turno de Guardia
 92          TurnoRequestDTO turnoReq1 = new
    TurnoRequestDTO();
 93          turnoReq1.setAdminId(admin.getId());
 94          turnoReq1.setEnfermeroId(enfermero.getId());
 95          turnoReq1.setAmbulanciaId(m04.getId());
 96          turnoReq1.setNombreChofer(chofer);
 97          turnoReq1.setNombreMedico(medico);
 98          turnoReq1.setFecha(LocalDate.now());
 99          turnoReq1.setHorario(HorarioTurno.MANIANA);
100
101          var turnoResp1 = turnoService.crearTurno(
    turnoReq1);
102          // Simular ingreso y egreso
103          var listaInsumosIngreso1 = insumos.stream().
    map(i -> new ControlInsumoRequestDTO(i.getId(),i.
    getPuntoControl(), null)).toList();
104          listaInsumosIngreso1.get(5).setCantidad(50);
105          turnoService.registrarIngreso(new
    IngresoEgresoDTO(turnoResp1.getId(),
    listaInsumosIngreso1, null));
106
107          var listaInsumosEgreso1 = List.copyOf(
    listaInsumosIngreso1);
108          listaInsumosEgreso1.get(0).setCantidad(
    listaInsumosEgreso1.get(0).getCantidad() - 1);
109          listaInsumosEgreso1.get(1).setCantidad(
```

```
109 listaInsumosEgreso1.get(1).getCantidad() - 2);
110         listaInsumosEgreso1.get(5).setCantidad(100);
111         turnoService.registrarEgreso(new
    IngresoEgresoDTO(turnoResp1.getId(),
    listaInsumosEgreso1, null));
112 
113         TurnoRequestDTO turnoReq2 = new
    TurnoRequestDTO();
114         turnoReq2.setAdminId(admin.getId());
115         turnoReq2.setEnfermeroId(enfermero.getId());
116         turnoReq2.setAmbulanciaId(m04.getId());
117         turnoReq2.setNombreChofer(chofer);
118         turnoReq2.setNombreMedico(medico);
119         turnoReq2.setFecha(LocalDate.now());
120         turnoReq2.setHorario(HorarioTurno.TARDE);
121 
122         var turnoResp2 = turnoService.crearTurno(
    turnoReq2);
123 
124         // Simular ingreso con inconsistencia en
    adrenalina
125         var listaInsumosIngreso2 = insumos.stream().
    map(i -> new ControlInsumoRequestDTO(i.getId(),i.
    getPuntoControl(),0)).toList();
126         listaInsumosIngreso2.get(0).setCantidad(
    listaInsumosIngreso2.get(0).getCantidad() - 10);
127 
128         turnoService.registrarIngreso(new
    IngresoEgresoDTO(turnoResp2.getId(),
    listaInsumosIngreso2, "Faltan 10 adrenalina"));
129     }
130 
131     private Ambulancia crearAmbulanciaConInsumos(
    String patente, Set<Insumo> insumosBase) {
132         Ambulancia amb = Ambulancia.builder()
133                 .patente(patente)
134                 .insumos(insumosBase)
135                 .build();
136         return ambulanciaRepository.save(amb);
137     }
138 
```

```
139 }
140
```

```java
package com.tup.tfi.control107.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import io.swagger.v3.oas.models.servers.Server;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.List;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        final String securitySchemeName = "bearerAuth";
        return new OpenAPI()
                .servers(List.of(new Server().url("/api").description("Default Server URL")))
                .addSecurityItem(new SecurityRequirement().addList(securitySchemeName))
                .components(new Components()
                        .addSecuritySchemes(securitySchemeName,
                                new SecurityScheme()
                                        .name(securitySchemeName)
                                        .type(SecurityScheme.Type.HTTP)
                                        .scheme("bearer")
                                        .bearerFormat("JWT")));
    }
}
```

```java
package com.tup.tfi.control107.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.annotation.web.configurers.HeadersConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) {
        return http
                .cors(Customizer.withDefaults())
                .csrf(AbstractHttpConfigurer::disable)
                .sessionManagement(session -> session
                        .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
                )
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/**").permitAll()
```

```
29                          .requestMatchers("/v3/api-
   docs/**", "/swagger-ui/**").permitAll()
30                          .requestMatchers("/h2-console
   /**").permitAll()
31                          .anyRequest().authenticated()
32                  )
33                  .headers(headers ->
34                          headers.frameOptions(
   HeadersConfigurer.FrameOptionsConfig::sameOrigin)
35                  )
36                  .build();
37      }
38
39 }
40
41
```

```
1 # Placeholder
2
```

```java
package com.tup.tfi.control107.service;

import com.tup.tfi.control107.dto.LoginDTO;
import com.tup.tfi.control107.dto.request.
LoginRequestDTO;
import com.tup.tfi.control107.exception.
CredencialesInvalidasException;

public interface AuthService {
    LoginDTO login(LoginRequestDTO request) throws
CredencialesInvalidasException;
}

```

```java
package com.tup.tfi.control107.service;

import com.tup.tfi.control107.dto.ResumenOperativoDTO;
import com.tup.tfi.control107.dto.TurnoDTO;
import com.tup.tfi.control107.dto.request.IngresoEgresoDTO;
import com.tup.tfi.control107.dto.request.TurnoRequestDTO;

import java.util.List;

public interface TurnoService {

    /**
     * Resumen para mostrar en la vista principal.
     */
    ResumenOperativoDTO obtenerResumenOperativo();

    TurnoDTO crearTurno(TurnoRequestDTO request);

    void registrarIngreso(IngresoEgresoDTO request);

    void registrarEgreso(IngresoEgresoDTO request);

    List<TurnoDTO> obtenerTodos();

    TurnoDTO obtenerPorId(Long id);

    List<TurnoDTO> obtenerPorEnfermeroId(Long enfermeroId);

    List<TurnoDTO> obtenerTurnosActivos();

    TurnoDTO cancelarTurno(Long turnoId, String observaciones);

    TurnoDTO modificarTurno(Long turnoId, TurnoRequestDTO request);
}

```

```java
package com.tup.tfi.control107.service;

import com.tup.tfi.control107.dto.InsumoDTO;
import com.tup.tfi.control107.dto.request.
InsumoRequestDTO;

import java.util.List;

public interface InsumoService {
    List<InsumoDTO> listarTodos();

    List<InsumoDTO> listarCriticos();

    InsumoDTO obtenerPorId(Long id);

    InsumoDTO crearInsumo(InsumoRequestDTO request);

    InsumoDTO actualizarInsumo(Long id,
InsumoRequestDTO request);

    void eliminarPorId(Long id);

    List<InsumoDTO> listarPorCategoria(Long
categoriaId);
}

```

```java
package com.tup.tfi.control107.service;

import com.tup.tfi.control107.dto.UsuarioDTO;
import com.tup.tfi.control107.dto.request.
UsuarioRequestDTO;

import java.util.List;

public interface UsuarioService {
    List<UsuarioDTO> listarTodos();

    UsuarioDTO obtenerPorId(Long id);

    UsuarioDTO crearUsuario(UsuarioRequestDTO request
);

    UsuarioDTO actualizarUsuario(Long id,
UsuarioRequestDTO request);

    void eliminarPorId(Long id);
}

```

```java
package com.tup.tfi.control107.service;

import com.tup.tfi.control107.dto.AmbulanciaDTO;
import com.tup.tfi.control107.dto.request.
AmbulanciaRequestDTO;

import java.util.List;

public interface AmbulanciaService {
    List<AmbulanciaDTO> listarTodas();

    AmbulanciaDTO obtenerPorId(Long id);

    AmbulanciaDTO crearAmbulancia(
AmbulanciaRequestDTO request);

    AmbulanciaDTO actualizarAmbulancia(Long id,
AmbulanciaRequestDTO request);

    void eliminarPorId(Long id);
}

```

```java
package com.tup.tfi.control107.service;

import com.tup.tfi.control107.dto.CategoriaInsumoDTO;
import com.tup.tfi.control107.dto.request.
CategoriaInsumoRequestDTO;

import java.util.List;

public interface CategoriaInsumoService {
    List<CategoriaInsumoDTO> listarTodas();

    CategoriaInsumoDTO obtenerPorId(Long id);

    CategoriaInsumoDTO crearCategoria(
CategoriaInsumoRequestDTO request);

    CategoriaInsumoDTO actualizarCategoria(Long id,
CategoriaInsumoRequestDTO request);

    void eliminarPorId(Long id);
}

```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.model.entity.BaseEntity;
import jakarta.persistence.EntityNotFoundException;
import org.springframework.data.jpa.repository.JpaRepository;

import java.time.LocalDateTime;

/**
 * Clase abstracta para evitar repetir codigo en los servicios concretos.
 * @param <T> la entidad correspondiente.
 */
abstract class AbstractService<T extends BaseEntity> {

    private final Class<T> clazz;
    private final JpaRepository<T, Long> repository;

    protected AbstractService(JpaRepository<T, Long> repository, Class<T> clazz) {
        this.repository = repository;
        this.clazz = clazz;
    }

    protected T obtener(Long id) {
        return repository.findById(id).filter(entity -> !entity.isEliminado()).orElseThrow(
                () -> new EntityNotFoundException(
                        String.format("No se encontro %s con id %d", clazz.getSimpleName(), id)
                )
        );
    }

    protected void eliminar(Long id) {
        T entity = obtener(id);
        entity.borradoLogico();
        entity.setUpdatedAt(LocalDateTime.now());
```

```
35          repository.save(entity);
36      }
37
38  }
39
```

```java
package com.tup.tfi.control107.service.impl;


import com.tup.tfi.control107.util.JwtUtil;
import com.tup.tfi.control107.util.PasswordUtil;
import com.tup.tfi.control107.dto.LoginDTO;
import com.tup.tfi.control107.dto.UsuarioDTO;
import com.tup.tfi.control107.dto.request.LoginRequestDTO;
import com.tup.tfi.control107.exception.CredencialesInvalidasException;
import com.tup.tfi.control107.model.entity.Usuario;
import com.tup.tfi.control107.repository.UsuarioRepository;
import com.tup.tfi.control107.service.AuthService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;


@Service
@RequiredArgsConstructor
public class AuthServiceImpl implements AuthService {

    private final UsuarioRepository usuarioRepository;
    private final JwtUtil jwtUtil;

    private static final String TOKEN_TYPE = "Bearer";
    private static final String TOKEN_SCOPE = "message:read";

    @Override
    public LoginDTO login(LoginRequestDTO request) throws CredencialesInvalidasException {
        Usuario usuario = usuarioRepository
                .findByEmailAndEliminadoIsFalse(request.mail().toLowerCase().trim())
                .orElseThrow(CredencialesInvalidasException::new);
        verificarClave(request, usuario);
```

```
33          String accessToken = jwtUtil.createToken(new
    UsuarioDTO(usuario));
34          return new LoginDTO(
35                  accessToken,
36                  jwtUtil.getExpirationTime(),
37                  TOKEN_TYPE,
38                  TOKEN_SCOPE
39          );
40      }
41
42      private void verificarClave(LoginRequestDTO
    request, Usuario usuario) throws
    CredencialesInvalidasException {
43          if (!PasswordUtil.verificarClave(
44                  request.clave().toCharArray(),
45                  usuario.getClave().getSalt(),
46                  usuario.getClave().getHash().
    toCharArray())
47          ) {
48              throw new CredencialesInvalidasException
    ();
49          }
50      }
51 }
52
```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.dto.AmbulanciaDTO;
import com.tup.tfi.control107.dto.ResumenOperativoDTO;
import com.tup.tfi.control107.dto.TurnoDTO;
import com.tup.tfi.control107.dto.request.IngresoEgresoDTO;
import com.tup.tfi.control107.dto.request.ControlInsumoRequestDTO;
import com.tup.tfi.control107.dto.request.TurnoRequestDTO;
import com.tup.tfi.control107.model.entity.*;
import com.tup.tfi.control107.model.enums.EstadoTurno;
import com.tup.tfi.control107.model.enums.HorarioTurno;
import com.tup.tfi.control107.model.enums.RolUsuario;
import com.tup.tfi.control107.repository.AmbulanciaRepository;
import com.tup.tfi.control107.repository.TurnoRepository;
import com.tup.tfi.control107.repository.UsuarioRepository;
import com.tup.tfi.control107.service.TurnoService;
import jakarta.persistence.EntityNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;

@Service
public class TurnoServiceImpl extends AbstractService<Turno> implements TurnoService {

```

```
31      private final TurnoRepository repository;
32      private final AmbulanciaRepository
    ambulanciaRepository;
33      private final UsuarioRepository usuarioRepository
    ;
34
35      private static final String
    OBSERVACIONES_CANCELADO = "Turno cancelado";
36
37      public TurnoServiceImpl(TurnoRepository
    repository, AmbulanciaRepository ambulanciaRepository
    , UsuarioRepository usuarioRepository) {
38          super(repository, Turno.class);
39          this.repository = repository;
40          this.ambulanciaRepository =
    ambulanciaRepository;
41          this.usuarioRepository = usuarioRepository;
42      }
43
44
45      @Transactional(readOnly = true)
46      @Override
47      public ResumenOperativoDTO
    obtenerResumenOperativo() {
48          var turnosActivos = obtenerActivos();
49          var ambulancias = ambulanciaRepository.
    findAllByEliminadoIsFalse();
50
51          return ResumenOperativoDTO.builder()
52                  .turnosActivos(turnosActivos.size())
53                  .ambulanciasDisponibles(ambulancias.
    stream().filter(Ambulancia::isDisponible).count())
54                  .ambulanciasTotales(ambulancias.size
    ())
55                  .alertas(turnosActivos.stream()
56                          .filter(t -> t.
    getControlInsumos().stream()
57                                  .anyMatch(i -> i.
    getIngresoFaltante() > 0 && i.getInsumo().isCritico
    ()))
58                          .count()
```

```
59                  )
60                  .turnosConInconsistencias(
   turnosActivos.stream()
61                          .filter(t -> t.
   getControlInsumos().stream()
62                                  .anyMatch(i -> i.
   getIngresoFaltante() > 0))
63                          .map(TurnoDTO::new).toList())
64                  .listaAmbulancias(ambulancias.stream
   ().map(AmbulanciaDTO::new).toList())
65                  .build();
66      }
67
68      @Transactional
69      @Override
70      public TurnoDTO crearTurno(TurnoRequestDTO
   request) {
71          Usuario admin = obtenerUsuario(request.
   getAdminId(), RolUsuario.ADMINISTRADOR);
72          Usuario enfermero = obtenerUsuario(request.
   getEnfermeroId(), RolUsuario.ENFERMERO);
73          Ambulancia ambulancia = obtenerAmbulancia(
   request.getAmbulanciaId());
74          validarDisponibilidad(request.getAmbulanciaId
   (), request.getEnfermeroId(), request.getFecha(),
   request.getHorario(), null);
75          Turno t = request.toTurno(admin, enfermero,
   ambulancia);
76          repository.save(t);
77          return new TurnoDTO(t);
78      }
79
80      @Transactional
81      @Override
82      public void registrarIngreso(IngresoEgresoDTO
   request) {
83          Turno t = obtener(request.getTurnoId());
84          if (!EstadoTurno.PROGRAMADO.equals(t.
   getEstado())) {
85              throw new IllegalArgumentException(
86                      String.format("No es posible
```

```
 86 registrar ingreso de turno %d con estado %s",
 87                                 request.getTurnoId(), t.
    getEstado()
 88                         ));
 89         }
 90         var mapaControl = verificarInsumos(request.
    getInsumos(), t);
 91         for (ControlInsumoRequestDTO dto : request.
    getInsumos()) {
 92             ControlInsumo control = mapaControl.get(
    dto.getId());
 93             control.informarIngreso(dto.getCantidad
    ());
 94         }
 95         t.setObservacionesIngreso(request.
    getObservaciones());
 96         t.setEstado(EstadoTurno.EN_CURSO);
 97         t.setUpdatedAt(LocalDateTime.now());
 98         repository.save(t);
 99     }
100
101     @Transactional
102     @Override
103     public void registrarEgreso(IngresoEgresoDTO
    request) {
104         Turno t = obtener(request.getTurnoId());
105         if (!EstadoTurno.EN_CURSO.equals(t.getEstado
    ())) {
106             throw new IllegalArgumentException(
107                     String.format("No es posible
    registrar egreso de turno %d con estado %s",
108                                 request.getTurnoId(), t.
    getEstado()
109                         ));
110         }
111         var mapaControl = verificarInsumos(request.
    getInsumos(), t);
112         for (ControlInsumoRequestDTO dto : request.
    getInsumos()) {
113             ControlInsumo control = mapaControl.get(
    dto.getId());
```

```
114             control.informarEgreso(
115                     dto.getCantidad(),
116                     null != dto.getCantidadRestockeo
    () ? dto.getCantidadRestockeo() : 0
117             );
118         }
119         t.setObservacionesEgreso(request.
    getObservaciones());
120         t.setEstado(EstadoTurno.FINALIZADO);
121         t.setUpdatedAt(LocalDateTime.now());
122         repository.save(t);
123     }
124 
125     @Override
126     public List<TurnoDTO> obtenerTodos() {
127         return repository.findAllByEliminadoIsFalse
    ().stream().map(TurnoDTO::new).toList();
128     }
129 
130     @Override
131     public TurnoDTO obtenerPorId(Long id) {
132         Turno t = obtener(id);
133         return new TurnoDTO(t);
134     }
135 
136     @Override
137     public List<TurnoDTO> obtenerPorEnfermeroId(Long
     enfermeroId) {
138         return repository.
    findAllByEnfermeroIdAndEliminadoIsFalseOrderByFechaD
    esc(enfermeroId).stream().map(TurnoDTO::new).toList
    ();
139     }
140 
141     @Override
142     public List<TurnoDTO> obtenerTurnosActivos() {
143         return obtenerActivos().stream().map(
    TurnoDTO::new).toList();
144     }
145 
146     @Transactional
```

```
147     @Override
148     public TurnoDTO cancelarTurno(Long turnoId,
    String observaciones) {
149         Turno t = obtener(turnoId);
150         if (!EstadoTurno.PROGRAMADO.equals(t.
    getEstado())) {
151             throw new IllegalArgumentException(
152                     String.format("No es posible
    cancelar turno %d con estado %s",
153                             turnoId, t.getEstado()
154                     ));
155         }
156         t.setEstado(EstadoTurno.CANCELADO);
157         t.setObservacionesEgreso(null !=
    observaciones ? observaciones :
    OBSERVACIONES_CANCELADO);
158         t.setUpdatedAt(LocalDateTime.now());
159         repository.save(t);
160         return new TurnoDTO(t);
161     }
162
163     @Transactional
164     @Override
165     public TurnoDTO modificarTurno(Long turnoId,
    TurnoRequestDTO request) {
166         Turno t = obtener(turnoId);
167         if (!EstadoTurno.PROGRAMADO.equals(t.
    getEstado())) {
168             throw new IllegalArgumentException(
169                     String.format("No es posible
    modificar turno %d con estado %s",
170                             turnoId, t.getEstado()
171                     ));
172         }
173         Ambulancia ambulancia = request.
    getAmbulanciaId() != null ? obtenerAmbulancia(
    request.getAmbulanciaId()) : null;
174         Usuario enfermero = request.getEnfermeroId
    () != null ? obtenerUsuario(request.getEnfermeroId
    (), RolUsuario.ENFERMERO) : null;
175
```

```
176         Long targetAmbulanciaId = null != ambulancia
     ? ambulancia.getId() : (null != t.getAmbulancia
    () ? t.getAmbulancia().getId() : null);
177         Long targetEnfermeroId = null != enfermero
     ? enfermero.getId() : (null != t.getEnfermero() ? t
    .getEnfermero().getId() : null);
178         LocalDate targetFecha = null != request.
    getFecha() ? request.getFecha() : t.getFecha();
179         HorarioTurno targetHorario = null != request
    .getHorario() ? request.getHorario() : t.getHorario
    ();
180
181         validarDisponibilidad(targetAmbulanciaId,
    targetEnfermeroId, targetFecha, targetHorario,
    turnoId);
182
183         request.aplicar(t, ambulancia, enfermero);
184         repository.save(t);
185         return new TurnoDTO(t);
186     }
187
188     private void validarDisponibilidad(Long
    ambulanciaId, Long enfermeroId, LocalDate fecha, com
    .tup.tfi.control107.model.enums.HorarioTurno horario
    , Long turnoIdExcluido) {
189         if (null != ambulanciaId && null != fecha
     && null != horario) {
190             boolean ocupada = (turnoIdExcluido ==
    null)
191                     ? repository.
    existsByAmbulanciaIdAndFechaAndHorarioAndEstadoNotAn
    dEliminadoIsFalse(
192                     ambulanciaId, fecha, horario,
    EstadoTurno.CANCELADO)
193                     : repository.
    existsByAmbulanciaIdAndFechaAndHorarioAndEstadoNotAn
    dEliminadoIsFalseAndIdNot(
194                     ambulanciaId, fecha, horario,
    EstadoTurno.CANCELADO, turnoIdExcluido);
195
196             if (ocupada) {
```

```
197                 throw new IllegalArgumentException("
    La ambulancia ya se encuentra asignada a un turno en
     la misma fecha y franja horaria.");
198             }
199         }
200 
201         if (null != enfermeroId && null != fecha &&
    null != horario) {
202             boolean ocupado = (turnoIdExcluido ==
    null)
203                     ? repository.
    existsByEnfermeroIdAndFechaAndHorarioAndEstadoNotAnd
    EliminadoIsFalse(
204                     enfermeroId, fecha, horario,
    EstadoTurno.CANCELADO)
205                     : repository.
    existsByEnfermeroIdAndFechaAndHorarioAndEstadoNotAnd
    EliminadoIsFalseAndIdNot(
206                     enfermeroId, fecha, horario,
    EstadoTurno.CANCELADO, turnoIdExcluido);
207 
208             if (ocupado) {
209                 throw new IllegalArgumentException("
    El enfermero ya se encuentra asignado a un turno en
    la misma fecha y franja horaria.");
210             }
211         }
212     }
213 
214     /**
215      * Mapea los controles existentes del turno por
    id, extrae el conjunto de IDs recibidos por request
216      * y valida que no contenga duplicados y
    coincidencia exacta con los IDs de control del Turno
    .
217      *
218      * @param insumos Lista de Ids de insumos y
    cantidades
219      * @return Mapa de Ids de insumos a registros de
     control
220      */
```

```
221     private Map<Long, ControlInsumo>
    verificarInsumos(List<ControlInsumoRequestDTO>
    insumos, Turno t) {
222         if (null == insumos || insumos.isEmpty()) {
223             throw new IllegalArgumentException("
    Lista de insumos para control de ingreso/egreso
    vacia o nula.");
224         }
225         Map<Long, ControlInsumo> mapaControl = t.
    getControlInsumos().stream()
226                 .collect(Collectors.toMap(c -> c.
    getInsumo().getId(), c -> c));
227         Set<Long> idsRecibidos = insumos.stream()
228                 .map(ControlInsumoRequestDTO::getId)
229                 .collect(Collectors.toSet());
230         if (idsRecibidos.size() != insumos.size()) {
231             throw new IllegalArgumentException("La
    lista enviada contiene insumos duplicados.");
232         }
233         if (!mapaControl.keySet().equals(
    idsRecibidos)) {
234             throw new IllegalArgumentException("Los
    insumos enviados no coinciden con el control del
    Turno.");
235         }
236         return mapaControl;
237     }
238
239     private List<Turno> obtenerActivos() {
240         return repository.
    findByEstadoAndEliminadoIsFalse(EstadoTurno.EN_CURSO
    );
241     }
242
243     private Ambulancia obtenerAmbulancia(Long
    ambulanciaId) {
244         return ambulanciaRepository.
    findByIdAndEliminadoIsFalse(ambulanciaId).
    orElseThrow(
245                 () -> new EntityNotFoundException("
    Ambulancia no encontrada")
```

```
246         );
247     }
248 
249     private Usuario obtenerUsuario(Long usuarioId,
    RolUsuario rol) {
250         Usuario encontrado = usuarioRepository.
    findByIdAndEliminadoIsFalse(usuarioId).orElseThrow(
251                 () -> new EntityNotFoundException("
    Usuario no encontrado")
252         );
253         if (!encontrado.getRol().equals(rol)) {
254             throw new IllegalArgumentException("El
    usuario solicitado no tiene el rol {" + rol.name
    () + "}");
255         }
256         return encontrado;
257     }
258 
259 }
260 
```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.dto.InsumoDTO;
import com.tup.tfi.control107.dto.request.InsumoRequestDTO;
import com.tup.tfi.control107.model.entity.CategoriaInsumo;
import com.tup.tfi.control107.model.entity.Insumo;
import com.tup.tfi.control107.repository.CategoriaRepository;
import com.tup.tfi.control107.repository.InsumoRepository;
import com.tup.tfi.control107.service.InsumoService;
import jakarta.persistence.EntityNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
public class InsumoServiceImpl extends AbstractService<Insumo> implements InsumoService {

    private final InsumoRepository repository;
    private final CategoriaRepository categoriaRepository;

    private final static int NOMBRE_MIN_LENGTH = 3;
    private final static int NOMBRE_MAX_LENGTH = 255;

    public InsumoServiceImpl(InsumoRepository repository, CategoriaRepository categoriaRepository) {
        super(repository, Insumo.class);
        this.repository = repository;
        this.categoriaRepository = categoriaRepository;
    }

    @Override
```

```
32      public List<InsumoDTO> listarTodos() {
33          return repository.findAllByEliminadoIsFalse
    ().stream().map(InsumoDTO::new).toList();
34      }
35
36      @Override
37      public List<InsumoDTO> listarCriticos() {
38          return repository.
    findByCriticoIsTrueAndEliminadoIsFalse().stream().map
    (InsumoDTO::new).toList();
39      }
40
41      @Override
42      public List<InsumoDTO> listarPorCategoria(Long
    categoriaId) {
43          return repository.
    findByCategoriaIdAndEliminadoIsFalse(categoriaId).
    stream().map(InsumoDTO::new).toList();
44      }
45
46      @Override
47      public InsumoDTO obtenerPorId(Long id) {
48          Insumo i = obtener(id);
49          return new InsumoDTO(i);
50      }
51
52      @Transactional
53      @Override
54      public InsumoDTO crearInsumo(InsumoRequestDTO
    request) {
55          validar(request, null);
56          var categoria = obtenerCategoria(request.
    getCategoriaId());
57          Insumo guardado = repository.save(request.
    toInsumo(categoria));
58          return new InsumoDTO(guardado);
59      }
60
61      @Transactional
62      @Override
63      public InsumoDTO actualizarInsumo(Long id,
```

```
63 InsumoRequestDTO request) {
64         validar(request, id);
65         Insumo i = obtener(id);
66         CategoriaInsumo categoria = request.
   getCategoriaId() != null ? obtenerCategoria(request.
   getCategoriaId()) : null;
67         request.aplicar(i, categoria);
68         repository.save(i);
69         return new InsumoDTO(i);
70     }
71 
72     @Transactional
73     @Override
74     public void eliminarPorId(Long id) {
75         eliminar(id);
76     }
77 
78     private void validar(InsumoRequestDTO request,
   Long id) {
79         if (request.getNombre() != null) {
80             String nombre = request.getNombre().trim
   ();
81             if (nombre.length() < NOMBRE_MIN_LENGTH
    || nombre.length() > NOMBRE_MAX_LENGTH) {
82                 throw new IllegalArgumentException("
   El nombre debe tener entre " + NOMBRE_MIN_LENGTH +
   " y " + NOMBRE_MAX_LENGTH + " caracteres");
83             }
84             boolean existe = (id == null)
85                     ? repository.
   existsByNombreIgnoreCaseAndEliminadoIsFalse(nombre)
86                     : repository.
   existsByNombreIgnoreCaseAndEliminadoIsFalseAndIdNot(
   nombre, id);
87             if (existe) {
88                 throw new IllegalArgumentException("
   Ya existe un insumo con ese nombre");
89             }
90         }
91         if (request.getPuntoControl() <= 0) {
92             throw new IllegalArgumentException("El
```

```
 92 punto de control debe ser mayor a 0");
 93         }
 94     }
 95
 96     private CategoriaInsumo obtenerCategoria(Long id
    ) {
 97         return categoriaRepository.
    findByIdAndEliminadoIsFalse(id).orElseThrow(
 98                 () -> new EntityNotFoundException("
    No se encontro categoria con id " + id)
 99         );
100     }
101 }
102
```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.util.PasswordUtil;
import com.tup.tfi.control107.dto.UsuarioDTO;
import com.tup.tfi.control107.dto.request.
UsuarioRequestDTO;
import com.tup.tfi.control107.model.entity.Usuario;
import com.tup.tfi.control107.repository.
UsuarioRepository;
import com.tup.tfi.control107.service.UsuarioService;
import org.apache.commons.lang3.StringUtils;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.
Transactional;

import java.util.List;

@Service
public class UsuarioServiceImpl extends
AbstractService<Usuario> implements UsuarioService {

    private final UsuarioRepository repository;

    private final static String REGEX_DNI = "^[0-9]{8
}$";

    public UsuarioServiceImpl(UsuarioRepository
repository) {
        super(repository, Usuario.class);
        this.repository = repository;
    }

    @Override
    public List<UsuarioDTO> listarTodos() {
        return repository.findAllByEliminadoIsFalse
().stream().map(UsuarioDTO::new).toList();
    }

    @Override
    public UsuarioDTO obtenerPorId(Long id) {
        Usuario u = obtener(id);
```

```
35          return new UsuarioDTO(u);
36      }
37
38      @Transactional
39      @Override
40      public UsuarioDTO crearUsuario(UsuarioRequestDTO
    request) {
41          validar(request, null);
42          var clave = PasswordUtil.hash(request.
    getClave().toCharArray(), null);
43          Usuario guardado = repository.save(request.
    toUsuario(clave));
44          return new UsuarioDTO(guardado);
45      }
46
47      @Transactional
48      @Override
49      public UsuarioDTO actualizarUsuario(Long id,
    UsuarioRequestDTO request) {
50          validar(request, id);
51          Usuario u = obtener(id);
52          request.aplicar(u);
53          repository.save(u);
54          return new UsuarioDTO(u);
55      }
56
57      @Transactional
58      @Override
59      public void eliminarPorId(Long id) {
60          eliminar(id);
61      }
62
63      private void validar(UsuarioRequestDTO request,
    Long id) {
64          if (request.getDni() != null) {
65              String dni = request.getDni().trim();
66              if (!StringUtils.leftPad(dni,8, "0").
    matches(REGEX_DNI)) {
67                  throw new IllegalArgumentException("
    Formato de DNI inválido");
68              }
```

```
69              boolean existeDni = (id == null)
70                      ? repository.
   existsByDniAndEliminadoIsFalse(dni)
71                      : repository.
   existsByDniAndEliminadoIsFalseAndIdNot(dni, id);
72              if (existeDni) {
73                  throw new IllegalArgumentException("
   Ya existe un usuario con ese DNI");
74              }
75          }
76          if (request.getEmail() != null) {
77              String email = request.getEmail().trim
   ();
78              boolean existeEmail = (id == null)
79                      ? repository.
   existsByEmailIgnoreCaseAndEliminadoIsFalse(email)
80                      : repository.
   existsByEmailIgnoreCaseAndEliminadoIsFalseAndIdNot(
   email, id);
81              if (existeEmail) {
82                  throw new IllegalArgumentException("
   Ya existe un usuario con ese email");
83              }
84          }
85      }
86 }
87
```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.dto.AmbulanciaDTO;
import com.tup.tfi.control107.dto.request.AmbulanciaRequestDTO;
import com.tup.tfi.control107.model.entity.Ambulancia;
import com.tup.tfi.control107.model.entity.Insumo;
import com.tup.tfi.control107.repository.AmbulanciaRepository;
import com.tup.tfi.control107.repository.InsumoRepository;
import com.tup.tfi.control107.service.AmbulanciaService;
import jakarta.persistence.EntityNotFoundException;
import org.jspecify.annotations.NonNull;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.Arrays;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

@Service
public class AmbulanciaServiceImpl extends AbstractService<Ambulancia> implements AmbulanciaService {

    private final AmbulanciaRepository repository;
    private final InsumoRepository insumoRepository;

    private final static String[] REGEX_PATENTE = {"^[A-Z]{3}\\d{3}$","^[A-Z]{2}\\d{3}[A-Z]{2}$"};

    public AmbulanciaServiceImpl(AmbulanciaRepository ambulanciaRepository, InsumoRepository insumoRepository) {
        super(ambulanciaRepository, Ambulancia.class);
```

```
30          this.repository = ambulanciaRepository;
31          this.insumoRepository = insumoRepository;
32      }
33
34      @Override
35      public List<AmbulanciaDTO> listarTodas() {
36          return repository.findAllByEliminadoIsFalse
    ().stream().map(AmbulanciaDTO::new).toList();
37      }
38
39      @Override
40      public AmbulanciaDTO obtenerPorId(Long id) {
41          Ambulancia a = obtener(id);
42          return new AmbulanciaDTO(a);
43      }
44
45      @Transactional
46      @Override
47      public AmbulanciaDTO crearAmbulancia(
    AmbulanciaRequestDTO request) {
48          validar(request, null);
49          var insumos = obtenerInsumos(request);
50          Ambulancia guardada = repository.save(request
    .toAmbulancia(insumos));
51          return new AmbulanciaDTO(guardada);
52      }
53
54      @Transactional
55      @Override
56      public AmbulanciaDTO actualizarAmbulancia(Long id
    , AmbulanciaRequestDTO request) {
57          validar(request, id);
58          Ambulancia a = obtener(id);
59          var insumos = request.getInsumos() != null ?
    obtenerInsumos(request) : null;
60          request.aplicar(a, insumos);
61          repository.save(a);
62          return new AmbulanciaDTO(a);
63      }
64
65      @Transactional
```

```
66      @Override
67      public void eliminarPorId(Long id) {
68          eliminar(id);
69      }
70
71      private void validar(AmbulanciaRequestDTO
    request, Long id) {
72          if (request.getPatente() != null) {
73              String patente = request.getPatente().
    trim();
74              boolean regexMatch = Arrays.stream(
    REGEX_PATENTE).anyMatch(patente::matches);
75              if (!regexMatch) {
76                  throw new IllegalArgumentException("
    Formato de patente inválido");
77              }
78              boolean existe = (id == null)
79                      ? repository.
    existsByPatenteIgnoreCaseAndEliminadoIsFalse(patente
    )
80                      : repository.
    existsByPatenteIgnoreCaseAndEliminadoIsFalseAndIdNot
    (patente, id);
81              if (existe) {
82                  throw new IllegalArgumentException("
    Ya existe una ambulancia con esa patente");
83              }
84          }
85
86      }
87
88      private @NonNull Set<Insumo> obtenerInsumos(
    AmbulanciaRequestDTO request) {
89          return request.getInsumos().stream()
90                  .map(i -> insumoRepository.
    findByIdAndEliminadoIsFalse(i)
91                          .orElseThrow(() -> new
    EntityNotFoundException("No se encontró el insumo
    con id " + i)))
92                  .collect(Collectors.toSet());
93      }
```

```
94 }
95
```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.dto.CategoriaInsumoDTO;
import com.tup.tfi.control107.dto.request.CategoriaInsumoRequestDTO;
import com.tup.tfi.control107.model.entity.CategoriaInsumo;
import com.tup.tfi.control107.repository.CategoriaRepository;
import com.tup.tfi.control107.service.CategoriaInsumoService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
public class CategoriaInsumoServiceImpl extends AbstractService<CategoriaInsumo> implements CategoriaInsumoService {

    private final CategoriaRepository repository;

    private final static String REGEX_COLOR = "^#(?:[0-9a-fA-F]{3}){1,2}$";
    private final static int NOMBRE_MIN_LENGTH = 3;
    private final static int NOMBRE_MAX_LENGTH = 100;

    public CategoriaInsumoServiceImpl(CategoriaRepository repository) {
        super(repository, CategoriaInsumo.class);
        this.repository = repository;
    }

    @Override
    public List<CategoriaInsumoDTO> listarTodas() {
        return repository.findAllByEliminadoIsFalse().stream().map(CategoriaInsumoDTO::new).toList();
    }

```

```
32      @Override
33      public CategoriaInsumoDTO obtenerPorId(Long id) {
34          CategoriaInsumo c = obtener(id);
35          return new CategoriaInsumoDTO(c);
36      }
37
38      @Transactional
39      @Override
40      public CategoriaInsumoDTO crearCategoria(
    CategoriaInsumoRequestDTO request) {
41          validar(request, null);
42          CategoriaInsumo guardado = repository.save(
    request.toCategoriaInsumo());
43          return new CategoriaInsumoDTO(guardado);
44      }
45
46      @Transactional
47      @Override
48      public CategoriaInsumoDTO actualizarCategoria(
    Long id, CategoriaInsumoRequestDTO request) {
49          validar(request, id);
50          CategoriaInsumo c = obtener(id);
51          request.aplicar(c);
52          repository.save(c);
53          return new CategoriaInsumoDTO(c);
54      }
55
56      @Transactional
57      @Override
58      public void eliminarPorId(Long id) {
59          eliminar(id);
60      }
61
62      private void validar(CategoriaInsumoRequestDTO
    request, Long id) {
63          if (request.getNombre() != null) {
64              String nombre = request.getNombre().trim
    ();
65              if (nombre.length() < NOMBRE_MIN_LENGTH
     || nombre.length() > NOMBRE_MAX_LENGTH) {
66                  throw new IllegalArgumentException("
```

```
66 El nombre debe tener entre " + NOMBRE_MIN_LENGTH +
   " y " + NOMBRE_MAX_LENGTH + " caracteres");
67             }
68             boolean existe = (id == null)
69                     ? repository.
   existsByNombreIgnoreCaseAndEliminadoIsFalse(nombre)
70                     : repository.
   existsByNombreIgnoreCaseAndEliminadoIsFalseAndIdNot(
   nombre, id);
71             if (existe) {
72                 throw new IllegalArgumentException("
   Ya existe una categoría con ese nombre");
73             }
74         }
75         if (request.getColor() != null) {
76             String color = request.getColor().trim
   ();
77             if (!color.matches(REGEX_COLOR)) {
78                 throw new IllegalArgumentException("
   Formato de color no válido. Ejemplo: #FF0000");
79             }
80         }
81     }
82 }
83
```

```
1 # Placeholder
2
```

```java
package com.tup.tfi.control107.exception;

public class CredencialesInvalidasException extends Exception {
}

```

```
1 # Placeholder
2
```

```java
package com.tup.tfi.control107.controller;

import com.tup.tfi.control107.dto.LoginDTO;
import com.tup.tfi.control107.dto.request.LoginRequestDTO;
import com.tup.tfi.control107.exception.CredencialesInvalidasException;
import com.tup.tfi.control107.service.AuthService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/v1/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthService service;

    @PostMapping("/login")
    public ResponseEntity<LoginDTO> login(@RequestBody LoginRequestDTO request) throws CredencialesInvalidasException {
        return ResponseEntity.ok(service.login(request));
    }
}

```

```java
package com.tup.tfi.control107.controller;

import com.tup.tfi.control107.dto.ResumenOperativoDTO;
import com.tup.tfi.control107.dto.TurnoDTO;
import com.tup.tfi.control107.dto.request.CancelarTurnoDTO;
import com.tup.tfi.control107.dto.request.IngresoEgresoDTO;
import com.tup.tfi.control107.dto.request.TurnoRequestDTO;
import com.tup.tfi.control107.service.TurnoService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/v1/turnos")
@RequiredArgsConstructor
public class TurnoController {

    private final TurnoService turnoService;

    @GetMapping
    public ResponseEntity<List<TurnoDTO>> mostrarTurnos() {
        List<TurnoDTO> turnos = turnoService.obtenerTodos();
        return ResponseEntity.ok(turnos);
    }

    @GetMapping("/activos")
    public ResponseEntity<List<TurnoDTO>> obtenerTurnosActivos() {
        return ResponseEntity.ok(turnoService.obtenerTurnosActivos());
    }

    @GetMapping("/enfermero/{enfermeroId}")
```

```
34      public ResponseEntity<List<TurnoDTO>>
    obtenerTurnosPorEnfermero(@PathVariable Long
    enfermeroId) {
35          return ResponseEntity.ok(turnoService.
    obtenerPorEnfermeroId(enfermeroId));
36      }
37
38      @GetMapping("/{id}")
39      public ResponseEntity<TurnoDTO> obtenerPorId(@
    PathVariable Long id) {
40          return ResponseEntity.ok(turnoService.
    obtenerPorId(id));
41      }
42
43      @GetMapping("/resumen")
44      public ResponseEntity<ResumenOperativoDTO>
    obtenerResumenOperativo() {
45          ResumenOperativoDTO resumen = turnoService.
    obtenerResumenOperativo();
46          return ResponseEntity.ok(resumen);
47      }
48
49      @PostMapping
50      public ResponseEntity<TurnoDTO> crearTurno(@
    RequestBody TurnoRequestDTO request) {
51          TurnoDTO turno = turnoService.crearTurno(
    request);
52          return ResponseEntity.ok(turno);
53      }
54
55      @PutMapping("/{id}")
56      public ResponseEntity<TurnoDTO> modificarTurno(@
    PathVariable Long id, @RequestBody TurnoRequestDTO
    request) {
57          TurnoDTO turno = turnoService.modificarTurno(
    id, request);
58          return ResponseEntity.ok(turno);
59      }
60
61      @PostMapping("/ingreso")
62      public ResponseEntity<Void> registrarIngreso(@
```

```
62 RequestBody IngresoEgresoDTO request) {
63         turnoService.registrarIngreso(request);
64         return ResponseEntity.ok().build();
65     }
66 
67     @PostMapping("/egreso")
68     public ResponseEntity<Void> registrarEgreso(@
   RequestBody IngresoEgresoDTO request) {
69         turnoService.registrarEgreso(request);
70         return ResponseEntity.ok().build();
71     }
72 
73     @PostMapping("/{id}/cancelar")
74     public ResponseEntity<TurnoDTO> cancelarTurno(
75             @PathVariable Long id,
76             @RequestBody CancelarTurnoDTO request) {
77         TurnoDTO turno = turnoService.cancelarTurno(
   id, request.getObservaciones());
78         return ResponseEntity.ok(turno);
79     }
80 }
81 
```

```java
package com.tup.tfi.control107.controller;

import com.tup.tfi.control107.dto.InsumoDTO;
import com.tup.tfi.control107.dto.request.
InsumoRequestDTO;
import com.tup.tfi.control107.service.InsumoService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/v1/insumos")
@RequiredArgsConstructor
public class InsumoController {

    private final InsumoService insumoService;

    @GetMapping
    public ResponseEntity<List<InsumoDTO>>
listarTodos() {
        return ResponseEntity.ok(insumoService.
listarTodos());
    }

    @GetMapping("/{id}")
    public ResponseEntity<InsumoDTO> obtenerPorId(@
PathVariable Long id) {
        return ResponseEntity.ok(insumoService.
obtenerPorId(id));
    }

    @GetMapping("/criticos")
    public ResponseEntity<List<InsumoDTO>>
listarCriticos() {
        return ResponseEntity.ok(insumoService.
listarCriticos());
    }

    @GetMapping("/categoria/{categoriaId}")
```

```
35      public ResponseEntity<List<InsumoDTO>>
   listarPorCategoria(@PathVariable Long categoriaId) {
36          return ResponseEntity.ok(insumoService.
   listarPorCategoria(categoriaId));
37      }
38
39      @PostMapping
40      public ResponseEntity<InsumoDTO> crear(@
   RequestBody InsumoRequestDTO request) {
41          return ResponseEntity.ok(insumoService.
   crearInsumo(request));
42      }
43
44      @PutMapping("/{id}")
45      public ResponseEntity<InsumoDTO> actualizar(@
   PathVariable Long id, @RequestBody InsumoRequestDTO
   request) {
46          return ResponseEntity.ok(insumoService.
   actualizarInsumo(id, request));
47      }
48
49      @DeleteMapping("/{id}")
50      public ResponseEntity<Void> eliminar(@
   PathVariable Long id) {
51          insumoService.eliminarPorId(id);
52          return ResponseEntity.noContent().build();
53      }
54  }
55
```

```java
package com.tup.tfi.control107.controller;

import com.tup.tfi.control107.dto.UsuarioDTO;
import com.tup.tfi.control107.dto.request.
UsuarioRequestDTO;
import com.tup.tfi.control107.service.UsuarioService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/v1/usuarios")
@RequiredArgsConstructor
public class UsuarioController {

    private final UsuarioService usuarioService;

    @GetMapping
    public ResponseEntity<List<UsuarioDTO>>
listarTodos() {
        return ResponseEntity.ok(usuarioService.
listarTodos());
    }

    @GetMapping("/{id}")
    public ResponseEntity<UsuarioDTO> obtenerPorId(@
PathVariable Long id) {
        UsuarioDTO usuario = usuarioService.
obtenerPorId(id);
        return ResponseEntity.ok(usuario);
    }

    @PostMapping
    public ResponseEntity<UsuarioDTO> crear(@
RequestBody UsuarioRequestDTO request) {
        UsuarioDTO guardado = usuarioService.
crearUsuario(request);
        return ResponseEntity.ok(guardado);
    }
```

```
35
36      @PutMapping("/{id}")
37      public ResponseEntity<UsuarioDTO> actualizar(@
    PathVariable Long id, @RequestBody UsuarioRequestDTO
    request) {
38          UsuarioDTO guardado = usuarioService.
    actualizarUsuario(id, request);
39          return ResponseEntity.ok(guardado);
40      }
41
42      @DeleteMapping("/{id}")
43      public ResponseEntity<Void> eliminar(@
    PathVariable Long id) {
44          usuarioService.eliminarPorId(id);
45          return ResponseEntity.noContent().build();
46      }
47  }
48
```

```java
package com.tup.tfi.control107.controller;

import com.tup.tfi.control107.dto.AmbulanciaDTO;
import com.tup.tfi.control107.dto.request.AmbulanciaRequestDTO;
import com.tup.tfi.control107.service.AmbulanciaService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/v1/ambulancias")
@RequiredArgsConstructor
public class AmbulanciaController {

    private final AmbulanciaService ambulanciaService;

    @GetMapping
    public ResponseEntity<List<AmbulanciaDTO>> listarTodas() {
        var dtos = ambulanciaService.listarTodas();
        return ResponseEntity.ok(dtos);
    }

    @GetMapping("/{id}")
    public ResponseEntity<AmbulanciaDTO> obtenerPorId(@PathVariable Long id) {
        var ambulancia = ambulanciaService.obtenerPorId(id);
        return ambulancia != null ? ResponseEntity.ok(ambulancia) : ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<AmbulanciaDTO> guardar(@RequestBody AmbulanciaRequestDTO ambulancia) {
        var guardada = ambulanciaService.
```

```
33 crearAmbulancia(ambulancia);
34         return ResponseEntity.ok(guardada);
35     }
36 
37     @PutMapping("/{id}")
38     public ResponseEntity<AmbulanciaDTO> actualizar(@
   PathVariable Long id, @RequestBody
   AmbulanciaRequestDTO ambulancia) {
39         var guardada = ambulanciaService.
   actualizarAmbulancia(id, ambulancia);
40         return ResponseEntity.ok(guardada);
41     }
42 
43     @DeleteMapping("/{id}")
44     public ResponseEntity<Void> eliminar(@
   PathVariable Long id) {
45         ambulanciaService.eliminarPorId(id);
46         return ResponseEntity.noContent().build();
47     }
48 }
49 
```

```java
package com.tup.tfi.control107.controller;

import com.tup.tfi.control107.dto.CategoriaInsumoDTO;
import com.tup.tfi.control107.dto.request.
CategoriaInsumoRequestDTO;
import com.tup.tfi.control107.service.
CategoriaInsumoService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/v1/categorias")
@RequiredArgsConstructor
public class CategoriaInsumoController {

    private final CategoriaInsumoService
categoriaService;

    @GetMapping
    public ResponseEntity<List<CategoriaInsumoDTO>>
listarTodas() {
        return ResponseEntity.ok(categoriaService.
listarTodas());
    }

    @GetMapping("/{id}")
    public ResponseEntity<CategoriaInsumoDTO>
obtenerPorId(@PathVariable Long id) {
        CategoriaInsumoDTO categoria =
categoriaService.obtenerPorId(id);
        return ResponseEntity.ok(categoria);
    }

    @PostMapping
    public ResponseEntity<CategoriaInsumoDTO> crear(@
RequestBody CategoriaInsumoRequestDTO request) {
        CategoriaInsumoDTO guardada =
categoriaService.crearCategoria(request);
```

```
33          return ResponseEntity.ok(guardada);
34      }
35
36      @PutMapping("/{id}")
37      public ResponseEntity<CategoriaInsumoDTO>
    actualizar(@PathVariable Long id, @RequestBody
    CategoriaInsumoRequestDTO request) {
38          CategoriaInsumoDTO guardada =
    categoriaService.actualizarCategoria(id, request);
39          return ResponseEntity.ok(guardada);
40      }
41
42      @DeleteMapping("/{id}")
43      public ResponseEntity<Void> eliminar(@
    PathVariable Long id) {
44          categoriaService.eliminarPorId(id);
45          return ResponseEntity.noContent().build();
46      }
47  }
48
```

```java
package com.tup.tfi.control107.controller.advice;

import com.tup.tfi.control107.dto.ErrorDTO;
import com.tup.tfi.control107.exception.
CredencialesInvalidasException;
import jakarta.persistence.EntityNotFoundException;
import jakarta.validation.
ConstraintViolationException;
import lombok.extern.slf4j.Slf4j;
import org.springframework.dao.
DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authorization.
AuthorizationDeniedException;
import org.springframework.web.bind.
MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.
ControllerAdvice;
import org.springframework.web.bind.annotation.
ExceptionHandler;

@Slf4j
@ControllerAdvice
public class GlobalExceptionHandler {

    private static final String MENSAJE_GENERICO = "
Ocurrió un error inesperado. Intente nuevamente en
unos minutos.";

    @ExceptionHandler(MethodArgumentNotValidException
.class)
    public ResponseEntity<ErrorDTO>
handleValidationError(MethodArgumentNotValidException
 ex) {
        var detalles = ex.getBindingResult()
                .getFieldErrors()
                .stream()
                .map(e -> e.getField() + ": " + e.
getDefaultMessage())
                .toList();
```

```
29
30          return ResponseEntity
31                  .status(HttpStatus.BAD_REQUEST)
32                  .body(ErrorDTO.of(HttpStatus.
   BAD_REQUEST.value(), "Error de validación", detalles
   ));
33      }
34
35      @ExceptionHandler(ConstraintViolationException.
   class)
36      public ResponseEntity<ErrorDTO>
   handleConstraintViolation(
   ConstraintViolationException ex) {
37          var detalles = ex.getConstraintViolations()
38                  .stream()
39                  .map(e -> e.getPropertyPath() + ": "
    + e.getMessage())
40                  .toList();
41
42          return ResponseEntity
43                  .status(HttpStatus.BAD_REQUEST)
44                  .body(ErrorDTO.of(HttpStatus.
   BAD_REQUEST.value(), "Error de persistencia",
   detalles));
45      }
46
47      @ExceptionHandler(DataIntegrityViolationException
   .class)
48      public ResponseEntity<ErrorDTO>
   handleDataIntegrityViolation(
   DataIntegrityViolationException ex) {
49          log.error("Error de persistencia", ex);
50          return ResponseEntity
51                  .status(HttpStatus.BAD_REQUEST)
52                  .body(ErrorDTO.simple(HttpStatus.
   BAD_REQUEST.value(), "Error de persistencia",
   MENSAJE_GENERICO));
53      }
54
55      @ExceptionHandler(EntityNotFoundException.class)
56      public ResponseEntity<ErrorDTO>
```

```
56 handleEntityNotFound(EntityNotFoundException ex) {
57         return ResponseEntity
58                 .status(HttpStatus.NOT_FOUND)
59                 .body(ErrorDTO.simple(HttpStatus.
   NOT_FOUND.value(), "Recurso no encontrado", ex.
   getMessage()));
60     }
61
62     @ExceptionHandler(CredencialesInvalidasException.
   class)
63     public ResponseEntity<ErrorDTO>
   handleCredencialesInvalidas(
   CredencialesInvalidasException ex) {
64         return ResponseEntity
65                 .status(HttpStatus.UNAUTHORIZED)
66                 .body(ErrorDTO.simple(HttpStatus.
   UNAUTHORIZED.value(), "Credenciales inválidas", "
   Credenciales inválidas"));
67     }
68
69     @ExceptionHandler(AuthorizationDeniedException.
   class)
70     public ResponseEntity<ErrorDTO>
   handleAccesoNoAutorizado(AuthorizationDeniedException
    ex) {
71         return ResponseEntity
72                 .status(HttpStatus.UNAUTHORIZED)
73                 .body(ErrorDTO.simple(HttpStatus.
   UNAUTHORIZED.value(), "Acceso no autorizado", ex.
   getMessage()));
74     }
75
76     @ExceptionHandler({IllegalArgumentException.class
   , IllegalStateException.class})
77     public ResponseEntity<ErrorDTO>
   handleIllegalArgumentException(RuntimeException ex) {
78         return ResponseEntity
79                 .status(HttpStatus.BAD_REQUEST)
80                 .body(ErrorDTO.simple(HttpStatus.
   BAD_REQUEST.value(), "Error de validación", ex.
   getMessage()));
```

```
81      }
82
83
84      @ExceptionHandler(Exception.class)
85      public ResponseEntity<ErrorDTO>
    handleGenericException(Exception ex) {
86          log.error("Error interno", ex);
87          return ResponseEntity
88                  .status(HttpStatus.
    INTERNAL_SERVER_ERROR)
89                  .body(ErrorDTO.simple(HttpStatus.
    INTERNAL_SERVER_ERROR.value(), "Error interno",
    MENSAJE_GENERICO));
90      }
91
92  }
93
```

```java
package com.tup.tfi.control107.controller.filter;

import com.tup.tfi.control107.util.JwtUtil;
import io.jsonwebtoken.Claims;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.jspecify.annotations.NonNull;
import org.springframework.security.authentication.
UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.
SimpleGrantedAuthority;
import org.springframework.security.core.context.
SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.
OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

@Component
@RequiredArgsConstructor
public class JwtFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            @NonNull HttpServletResponse response,
            @NonNull FilterChain filterChain
    )
            throws ServletException, IOException {

        String authHeader = request.getHeader("
Authorization");
        if (authHeader == null || !authHeader.
startsWith("Bearer ")) {
```

```
36              filterChain.doFilter(request, response);
37              return;
38          }
39
40          String token = authHeader.substring(7);
41
42          if (jwtUtil.isTokenExpired(token)) {
43              response.sendError(HttpServletResponse.
    SC_UNAUTHORIZED, "Token expirado");
44              return;
45          }
46
47          Claims claims = jwtUtil.extractClaims(token);
48          SimpleGrantedAuthority authority = new
    SimpleGrantedAuthority("ROLE_" + claims.get("rol"));
49
50          UsernamePasswordAuthenticationToken auth =
    new UsernamePasswordAuthenticationToken(
51                  claims.getSubject(),
52                  null,
53                  List.of(authority));
54
55          SecurityContextHolder.getContext().
    setAuthentication(auth);
56
57          filterChain.doFilter(request, response);
58      }
59 }
60
```

```
1 # Placeholder
2
```

```java
package com.tup.tfi.control107.repository;

import com.tup.tfi.control107.model.entity.Turno;
import com.tup.tfi.control107.model.enums.EstadoTurno
;
import com.tup.tfi.control107.model.enums.
HorarioTurno;
import org.springframework.data.jpa.repository.
JpaRepository;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;

@Repository
public interface TurnoRepository extends
JpaRepository<Turno, Long> {
    List<Turno> findByEstadoAndEliminadoIsFalse(
EstadoTurno estado);
    List<Turno> findAllByEliminadoIsFalse();
    List<Turno>
findAllByEnfermeroIdAndEliminadoIsFalseOrderByFechaDe
sc(Long enfermeroId);

    boolean
existsByAmbulanciaIdAndFechaAndHorarioAndEstadoNotAnd
EliminadoIsFalse(
            Long ambulanciaId, LocalDate fecha,
HorarioTurno horario, EstadoTurno estadoNot
    );

    boolean
existsByAmbulanciaIdAndFechaAndHorarioAndEstadoNotAnd
EliminadoIsFalseAndIdNot(
            Long ambulanciaId, LocalDate fecha,
HorarioTurno horario, EstadoTurno estadoNot, Long
idNot
    );

    boolean
existsByEnfermeroIdAndFechaAndHorarioAndEstadoNotAndE
```

```
liminadoIsFalse(
            Long enfermeroId, LocalDate fecha,
HorarioTurno horario, EstadoTurno estadoNot
    );

    boolean
existsByEnfermeroIdAndFechaAndHorarioAndEstadoNotAndE
liminadoIsFalseAndIdNot(
            Long enfermeroId, LocalDate fecha,
HorarioTurno horario, EstadoTurno estadoNot, Long
idNot
    );
}

```

```java
package com.tup.tfi.control107.repository;

import com.tup.tfi.control107.model.entity.Insumo;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface InsumoRepository extends JpaRepository<Insumo, Long> {

    List<Insumo> findAllByEliminadoIsFalse();
    List<Insumo> findByCriticoIsTrueAndEliminadoIsFalse();
    Optional<Insumo> findByIdAndEliminadoIsFalse(Long i);
    List<Insumo> findByCategoriaIdAndEliminadoIsFalse(Long categoriaId);
    boolean existsByNombreIgnoreCaseAndEliminadoIsFalse(String nombre);
    boolean existsByNombreIgnoreCaseAndEliminadoIsFalseAndIdNot(String nombre, Long id);
}

```

```java
package com.tup.tfi.control107.repository;

import com.tup.tfi.control107.model.entity.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    List<Usuario> findAllByEliminadoIsFalse();
    Optional<Usuario> findByEmailAndEliminadoIsFalse(String email);
    Optional<Usuario> findByIdAndEliminadoIsFalse(Long usuarioId);
    boolean existsByDniAndEliminadoIsFalse(String dni);
    boolean existsByDniAndEliminadoIsFalseAndIdNot(String dni, Long id);
    boolean existsByEmailIgnoreCaseAndEliminadoIsFalse(String email);
    boolean existsByEmailIgnoreCaseAndEliminadoIsFalseAndIdNot(String email, Long id);
}

```

```java
package com.tup.tfi.control107.repository;

import com.tup.tfi.control107.model.entity.CategoriaInsumo;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface CategoriaRepository extends JpaRepository<CategoriaInsumo, Long> {
    List<CategoriaInsumo> findAllByEliminadoIsFalse();
    Optional<CategoriaInsumo> findByIdAndEliminadoIsFalse(Long id);
    boolean existsByNombreIgnoreCaseAndEliminadoIsFalse(String nombre);
    boolean existsByNombreIgnoreCaseAndEliminadoIsFalseAndIdNot(String nombre, Long id);
}

```

```java
package com.tup.tfi.control107.repository;

import com.tup.tfi.control107.model.entity.Ambulancia;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;


@Repository
public interface AmbulanciaRepository extends JpaRepository<Ambulancia, Long> {
    List<Ambulancia> findAllByEliminadoIsFalse();

    Optional<Ambulancia> findByIdAndEliminadoIsFalse(Long ambulanciaId);

    boolean existsByPatenteIgnoreCaseAndEliminadoIsFalse(String patente);

    boolean existsByPatenteIgnoreCaseAndEliminadoIsFalseAndIdNot(String patente, Long id);
}

```

```
1 # Placeholder
2
```

```yaml
spring:
  application:
    name: control107

  profiles:
    active: dev

  datasource:
    url: "jdbc:h2:file:./data/control107;DB_CLOSE_DELAY=-1"
    driverClassName: org.h2.Driver
    username: ${DB_USER:sa}
    password: ${DB_PASSWORD:}

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: false
    properties:
      hibernate:
        format_sql: true

  h2:
    console:
      enabled: true
      path: /h2-console

  security:
    user:
      name: ${SECURITY_USER_NAME:admin}
      password: ${SECURITY_USER_PWD:admin}

springdoc:
  api-docs:
    enabled: true
  swagger-ui:
    enabled: true
    operationsSorter: method

logging:
  level:
```

```
41     org.springframework: warn
42
43 app:
44   cors:
45     allowed-origins: "*"
46
47 server:
48   forward-headers-strategy: framework
49   servlet:
50     context-path: /api
51
52 management:
53   server:
54     port: 9001
55   endpoints:
56     web:
57       exposure:
58         include: "*"
```

```
1 # Placeholder
2
```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.dto.ResumenOperativoDTO;
import com.tup.tfi.control107.dto.TurnoDTO;
import com.tup.tfi.control107.dto.request.ControlInsumoRequestDTO;
import com.tup.tfi.control107.dto.request.IngresoEgresoDTO;
import com.tup.tfi.control107.dto.request.TurnoRequestDTO;
import com.tup.tfi.control107.model.entity.*;
import com.tup.tfi.control107.model.enums.EstadoTurno;
import com.tup.tfi.control107.model.enums.HorarioTurno;
import com.tup.tfi.control107.model.enums.RolUsuario;
import com.tup.tfi.control107.repository.AmbulanciaRepository;
import com.tup.tfi.control107.repository.TurnoRepository;
import com.tup.tfi.control107.repository.UsuarioRepository;
import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

```

```
33 @ExtendWith(MockitoExtension.class)
34 class TurnoServiceImplTest {
35
36     @Mock
37     private TurnoRepository repository;
38
39     @Mock
40     private AmbulanciaRepository ambulanciaRepository
   ;
41
42     @Mock
43     private UsuarioRepository usuarioRepository;
44
45     @InjectMocks
46     private TurnoServiceImpl turnoService;
47
48     private Usuario admin;
49     private Usuario enfermero;
50     private Ambulancia ambulancia;
51     private Insumo insumoCritico;
52     private Insumo insumoNormal;
53     private Turno turno;
54     private ControlInsumo controlCritico;
55     private ControlInsumo controlNormal;
56     private TurnoRequestDTO turnoRequestDTO;
57
58     @BeforeEach
59     void setUp() {
60         admin = Usuario.builder()
61                 .nombre("Admin Test")
62                 .dni("11111111")
63                 .email("admin@test.com")
64                 .rol(RolUsuario.ADMINISTRADOR)
65                 .activo(true)
66                 .build();
67         admin.setId(1L);
68
69         enfermero = Usuario.builder()
70                 .nombre("Enfermero Test")
71                 .dni("22222222")
72                 .email("enfermero@test.com")
```

```
 73                 .rol(RolUsuario.ENFERMERO)
 74                 .activo(true)
 75                 .build();
 76         enfermero.setId(2L);
 77
 78         CategoriaInsumo categoria = CategoriaInsumo.
    builder()
 79                 .nombre("MEDICACION")
 80                 .color("#FF0000")
 81                 .build();
 82         categoria.setId(10L);
 83
 84         insumoCritico = Insumo.builder()
 85                 .nombre("Oxigeno")
 86                 .categoria(categoria)
 87                 .puntoControl(50)
 88                 .critico(true)
 89                 .build();
 90         insumoCritico.setId(100L);
 91
 92         insumoNormal = Insumo.builder()
 93                 .nombre("Gasa")
 94                 .categoria(categoria)
 95                 .puntoControl(10)
 96                 .critico(false)
 97                 .build();
 98         insumoNormal.setId(101L);
 99
100         ambulancia = Ambulancia.builder()
101                 .patente("AE 107 AB")
102                 .disponible(true)
103                 .insumos(Set.of(insumoCritico,
    insumoNormal))
104                 .build();
105         ambulancia.setId(5L);
106
107         turno = Turno.builder()
108                 .fecha(LocalDate.now())
109                 .horario(HorarioTurno.MANIANA)
110                 .estado(EstadoTurno.PROGRAMADO)
111                 .ambulancia(ambulancia)
```

```
112                 .admin(admin)
113                 .enfermero(enfermero)
114                 .nombreChofer("Chofer Test")
115                 .nombreMedico("Medico Test")
116                 .controlInsumos(new ArrayList<>())
117                 .build();
118         turno.setId(20L);
119 
120         controlCritico = ControlInsumo.builder()
121                 .insumo(insumoCritico)
122                 .turno(turno)
123                 .cantidadIngreso(40)
124                 .ingresoFaltante(10)
125                 .cantidadRestockeo(0)
126                 .build();
127         controlCritico.setId(200L);
128 
129         controlNormal = ControlInsumo.builder()
130                 .insumo(insumoNormal)
131                 .turno(turno)
132                 .cantidadIngreso(10)
133                 .ingresoFaltante(0)
134                 .cantidadRestockeo(0)
135                 .build();
136         controlNormal.setId(201L);
137 
138         turno.getControlInsumos().add(controlCritico
    );
139         turno.getControlInsumos().add(controlNormal
    );
140 
141         turnoRequestDTO = new TurnoRequestDTO(
142                 LocalDate.now(),
143                 HorarioTurno.MANIANA,
144                 1L,
145                 2L,
146                 5L,
147                 "Chofer Test",
148                 "Medico Test"
149         );
150     }
```

```
151
152     @Test
153     void obtenerResumenOperativo_ok() {
154         turno.setEstado(EstadoTurno.EN_CURSO);
155         when(repository.
    findByEstadoAndEliminadoIsFalse(EstadoTurno.EN_CURSO
    )).thenReturn(List.of(turno));
156         when(ambulanciaRepository.
    findAllByEliminadoIsFalse()).thenReturn(List.of(
    ambulancia));
157
158         ResumenOperativoDTO resumen = turnoService.
    obtenerResumenOperativo();
159
160         assertNotNull(resumen);
161         assertEquals(1, resumen.getTurnosActivos());
162         assertEquals(1, resumen.
    getAmbulanciasDisponibles());
163         assertEquals(1, resumen.
    getAmbulanciasTotales());
164         assertEquals(1, resumen.getAlertas());
165         assertEquals(1, resumen.
    getTurnosConInconsistencias().size());
166         assertEquals(1, resumen.getListaAmbulancias
    ().size());
167         verify(repository, times(1)).
    findByEstadoAndEliminadoIsFalse(EstadoTurno.EN_CURSO
    );
168         verify(ambulanciaRepository, times(1)).
    findAllByEliminadoIsFalse();
169     }
170
171     @Test
172     void crearTurno_ok() {
173         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(1L)).thenReturn(Optional
    .of(admin));
174         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(2L)).thenReturn(Optional
    .of(enfermero));
175         when(ambulanciaRepository.
```

```
175 findByIdAndEliminadoIsFalse(5L)).thenReturn(Optional
    .of(ambulancia));
176         when(repository.save(any(Turno.class))).
    thenReturn(turno);
177 
178         TurnoDTO resultado = turnoService.crearTurno
    (turnoRequestDTO);
179 
180         assertNotNull(resultado);
181         assertEquals(EstadoTurno.PROGRAMADO,
    resultado.getEstado());
182         verify(usuarioRepository, times(1)).
    findByIdAndEliminadoIsFalse(1L);
183         verify(usuarioRepository, times(1)).
    findByIdAndEliminadoIsFalse(2L);
184         verify(ambulanciaRepository, times(1)).
    findByIdAndEliminadoIsFalse(5L);
185         verify(repository, times(1)).save(any(Turno.
    class));
186     }
187 
188     @Test
189     void crearTurno_errorAdminNoEncontrado() {
190         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(1L)).thenReturn(Optional
    .empty());
191 
192         assertThrows(EntityNotFoundException.class
    , () -> turnoService.crearTurno(turnoRequestDTO));
193         verify(usuarioRepository, times(1)).
    findByIdAndEliminadoIsFalse(1L);
194         verify(repository, never()).save(any(Turno.
    class));
195     }
196 
197     @Test
198     void crearTurno_errorEnfermeroNoEncontrado() {
199         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(1L)).thenReturn(Optional
    .of(admin));
200         when(usuarioRepository.
```

```
200 findByIdAndEliminadoIsFalse(2L)).thenReturn(Optional
    .empty());
201
202         assertThrows(EntityNotFoundException.class
    , () -> turnoService.crearTurno(turnoRequestDTO));
203         verify(usuarioRepository, times(1)).
    findByIdAndEliminadoIsFalse(1L);
204         verify(usuarioRepository, times(1)).
    findByIdAndEliminadoIsFalse(2L);
205         verify(repository, never()).save(any(Turno.
    class));
206     }
207
208     @Test
209     void crearTurno_errorAmbulanciaNoEncontrada() {
210         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(1L)).thenReturn(Optional
    .of(admin));
211         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(2L)).thenReturn(Optional
    .of(enfermero));
212         when(ambulanciaRepository.
    findByIdAndEliminadoIsFalse(5L)).thenReturn(Optional
    .empty());
213
214         assertThrows(EntityNotFoundException.class
    , () -> turnoService.crearTurno(turnoRequestDTO));
215         verify(usuarioRepository, times(1)).
    findByIdAndEliminadoIsFalse(1L);
216         verify(usuarioRepository, times(1)).
    findByIdAndEliminadoIsFalse(2L);
217         verify(ambulanciaRepository, times(1)).
    findByIdAndEliminadoIsFalse(5L);
218         verify(repository, never()).save(any(Turno.
    class));
219     }
220
221     @Test
222     void crearTurno_errorAmbulanciaOcupada() {
223         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(1L)).thenReturn(Optional
```

```
223 .of(admin));
224         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(2L)).thenReturn(Optional
    .of(enfermero));
225         when(ambulanciaRepository.
    findByIdAndEliminadoIsFalse(5L)).thenReturn(Optional
    .of(ambulancia));
226         when(repository.
    existsByAmbulanciaIdAndFechaAndHorarioAndEstadoNotAn
    dEliminadoIsFalse(
227                 eq(5L), any(LocalDate.class), any(
    HorarioTurno.class), eq(EstadoTurno.CANCELADO)
228         )).thenReturn(true);
229
230         assertThrows(IllegalArgumentException.class
    , () -> turnoService.crearTurno(turnoRequestDTO));
231         verify(repository, never()).save(any(Turno.
    class));
232     }
233
234     @Test
235     void crearTurno_errorEnfermeroOcupado() {
236         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(1L)).thenReturn(Optional
    .of(admin));
237         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(2L)).thenReturn(Optional
    .of(enfermero));
238         when(ambulanciaRepository.
    findByIdAndEliminadoIsFalse(5L)).thenReturn(Optional
    .of(ambulancia));
239         when(repository.
    existsByAmbulanciaIdAndFechaAndHorarioAndEstadoNotAn
    dEliminadoIsFalse(
240                 eq(5L), any(LocalDate.class), any(
    HorarioTurno.class), eq(EstadoTurno.CANCELADO)
241         )).thenReturn(false);
242         when(repository.
    existsByEnfermeroIdAndFechaAndHorarioAndEstadoNotAnd
    EliminadoIsFalse(
243                 eq(2L), any(LocalDate.class), any(
```

```
243 HorarioTurno.class), eq(EstadoTurno.CANCELADO)
244         )).thenReturn(true);
245 
246         assertThrows(IllegalArgumentException.class
    , () -> turnoService.crearTurno(turnoRequestDTO));
247         verify(repository, never()).save(any(Turno.
    class));
248     }
249 
250     @Test
251     void crearTurno_errorNoEsEnfermero() {
252         // Id incorrecto (es Admin)
253         turnoRequestDTO.setEnfermeroId(1L);
254         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(1L)).thenReturn(Optional
    .of(admin));
255         assertThrows(IllegalArgumentException.class
    , () -> turnoService.crearTurno(turnoRequestDTO));
256         verify(repository, never()).save(any(Turno.
    class));
257     }
258 
259     @Test
260     void registrarIngreso_ok() {
261         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
262         when(repository.save(any(Turno.class))).
    thenReturn(turno);
263 
264         IngresoEgresoDTO requestIngreso = new
    IngresoEgresoDTO(
265                 20L,
266                 List.of(
267                         new ControlInsumoRequestDTO(
    100L, 45, null),
268                         new ControlInsumoRequestDTO(
    101L, 10, null)
269                 ),
270                 "Ingreso sin observaciones"
271         );
272 
```

```
273         turnoService.registrarIngreso(requestIngreso
    );
274 
275         assertEquals(EstadoTurno.EN_CURSO, turno.
    getEstado());
276         assertEquals("Ingreso sin observaciones",
    turno.getObservacionesIngreso());
277         assertEquals(45, controlCritico.
    getCantidadIngreso());
278         assertEquals(5, controlCritico.
    getIngresoFaltante());
279         verify(repository, times(1)).findById(20L);
280         verify(repository, times(1)).save(turno);
281     }
282 
283     @Test
284     void registrarIngreso_errorEstadoNoProgramado
    () {
285         turno.setEstado(EstadoTurno.EN_CURSO);
286         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
287 
288         IngresoEgresoDTO requestIngreso = new
    IngresoEgresoDTO(20L, List.of(), "Obs");
289 
290         assertThrows(IllegalArgumentException.class
    , () -> turnoService.registrarIngreso(requestIngreso
    ));
291         verify(repository, times(1)).findById(20L);
292         verify(repository, never()).save(any(Turno.
    class));
293     }
294 
295     @Test
296     void registrarIngreso_errorInsumosNulosOVacios
    () {
297         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
298 
299         IngresoEgresoDTO requestIngresoNull = new
    IngresoEgresoDTO(20L, null, "Obs");
```

```
300         IngresoEgresoDTO requestIngresoVacio = new
    IngresoEgresoDTO(20L, List.of(), "Obs");
301
302         assertThrows(IllegalArgumentException.class
    , () -> turnoService.registrarIngreso(
    requestIngresoNull));
303         assertThrows(IllegalArgumentException.class
    , () -> turnoService.registrarIngreso(
    requestIngresoVacio));
304         verify(repository, never()).save(any(Turno.
    class));
305     }
306
307     @Test
308     void registrarIngreso_errorInsumosDuplicados() {
309         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
310
311         IngresoEgresoDTO requestDuplicados = new
    IngresoEgresoDTO(
312                 20L,
313                 List.of(
314                         new ControlInsumoRequestDTO(
    100L, 45, null),
315                         new ControlInsumoRequestDTO(
    100L, 45, null)
316                 ),
317                 "Obs"
318         );
319
320         assertThrows(IllegalArgumentException.class
    , () -> turnoService.registrarIngreso(
    requestDuplicados));
321         verify(repository, never()).save(any(Turno.
    class));
322     }
323
324     @Test
325     void
    registrarIngreso_errorInsumosNoCoincidenConTurno() {
326         when(repository.findById(20L)).thenReturn(
```

```
326 Optional.of(turno));
327
328         IngresoEgresoDTO requestInvalido = new
    IngresoEgresoDTO(
329                 20L,
330                 List.of(
331                         new ControlInsumoRequestDTO(
    100L, 45, null),
332                         new ControlInsumoRequestDTO(
    999L, 10, null)
333                 ),
334                 "Obs"
335         );
336
337         assertThrows(IllegalArgumentException.class
    , () -> turnoService.registrarIngreso(
    requestInvalido));
338         verify(repository, never()).save(any(Turno.
    class));
339     }
340
341     @Test
342     void registrarEgreso_ok() {
343         turno.setEstado(EstadoTurno.EN_CURSO);
344         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
345         when(repository.save(any(Turno.class))).
    thenReturn(turno);
346
347         IngresoEgresoDTO requestEgreso = new
    IngresoEgresoDTO(
348                 20L,
349                 List.of(
350                         new ControlInsumoRequestDTO(
    100L, 30, 10),
351                         new ControlInsumoRequestDTO(
    101L, 8, null)
352                 ),
353                 "Egreso completado"
354         );
355
```

```
356         turnoService.registrarEgreso(requestEgreso);
357 
358         assertEquals(EstadoTurno.FINALIZADO, turno.
    getEstado());
359         assertEquals("Egreso completado", turno.
    getObservacionesEgreso());
360         assertEquals(30, controlCritico.
    getCantidadEgreso());
361         assertEquals(10, controlCritico.
    getCantidadRestockeo());
362         assertEquals(20, controlCritico.
    getCantidadUsada());
363         verify(repository, times(1)).findById(20L);
364         verify(repository, times(1)).save(turno);
365     }
366 
367     @Test
368     void registrarEgreso_errorEstadoNoEnCurso() {
369         turno.setEstado(EstadoTurno.PROGRAMADO);
370         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
371 
372         IngresoEgresoDTO requestEgreso = new
    IngresoEgresoDTO(20L, List.of(), "Obs");
373 
374         assertThrows(IllegalArgumentException.class
    , () -> turnoService.registrarEgreso(requestEgreso
    ));
375         verify(repository, times(1)).findById(20L);
376         verify(repository, never()).save(any(Turno.
    class));
377     }
378 
379     @Test
380     void obtenerTodos_ok() {
381         when(repository.findAllByEliminadoIsFalse
    ()).thenReturn(List.of(turno));
382 
383         List<TurnoDTO> resultado = turnoService.
    obtenerTodos();
384 
```

```
385         assertNotNull(resultado);
386         assertEquals(1, resultado.size());
387         assertEquals(20L, resultado.get(0).getId());
388         verify(repository, times(1)).
    findAllByEliminadoIsFalse();
389     }
390 
391     @Test
392     void obtenerPorId_ok() {
393         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
394 
395         TurnoDTO resultado = turnoService.
    obtenerPorId(20L);
396 
397         assertNotNull(resultado);
398         assertEquals(20L, resultado.getId());
399         verify(repository, times(1)).findById(20L);
400     }
401 
402     @Test
403     void obtenerPorId_errorNoEncontrado() {
404         when(repository.findById(20L)).thenReturn(
    Optional.empty());
405 
406         assertThrows(EntityNotFoundException.class
    , () -> turnoService.obtenerPorId(20L));
407         verify(repository, times(1)).findById(20L);
408     }
409 
410     @Test
411     void obtenerTurnosActivos_ok() {
412         turno.setEstado(EstadoTurno.EN_CURSO);
413         when(repository.
    findByEstadoAndEliminadoIsFalse(EstadoTurno.EN_CURSO
    )).thenReturn(List.of(turno));
414 
415         List<TurnoDTO> resultado = turnoService.
    obtenerTurnosActivos();
416 
417         assertNotNull(resultado);
```

```
418         assertEquals(1, resultado.size());
419         assertEquals(EstadoTurno.EN_CURSO, resultado
    .get(0).getEstado());
420         verify(repository, times(1)).
    findByEstadoAndEliminadoIsFalse(EstadoTurno.EN_CURSO
    );
421     }
422 
423     @Test
424     void cancelarTurno_ok_conObservaciones() {
425         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
426         when(repository.save(any(Turno.class))).
    thenReturn(turno);
427 
428         TurnoDTO resultado = turnoService.
    cancelarTurno(20L, "Falta oxigeno urgente");
429 
430         assertNotNull(resultado);
431         assertEquals(EstadoTurno.CANCELADO, turno.
    getEstado());
432         assertEquals("Falta oxigeno urgente", turno.
    getObservacionesEgreso());
433         verify(repository, times(1)).findById(20L);
434         verify(repository, times(1)).save(turno);
435     }
436 
437     @Test
438     void cancelarTurno_ok_observacionesDefault() {
439         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
440         when(repository.save(any(Turno.class))).
    thenReturn(turno);
441 
442         TurnoDTO resultado = turnoService.
    cancelarTurno(20L, null);
443 
444         assertNotNull(resultado);
445         assertEquals(EstadoTurno.CANCELADO, turno.
    getEstado());
446         assertEquals("Turno cancelado", turno.
```

```
446 getObservacionesEgreso());
447         verify(repository, times(1)).findById(20L);
448         verify(repository, times(1)).save(turno);
449     }
450 
451     @Test
452     void cancelarTurno_errorEstadoNoProgramado() {
453         turno.setEstado(EstadoTurno.EN_CURSO);
454         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
455 
456         assertThrows(IllegalArgumentException.class
    , () -> turnoService.cancelarTurno(20L, "Obs"));
457         verify(repository, times(1)).findById(20L);
458         verify(repository, never()).save(any(Turno.
    class));
459     }
460 
461     @Test
462     void modificarTurno_ok_conAmbulanciaYEnfermero
    () {
463         Turno turnoSinControl = Turno.builder()
464                 .fecha(LocalDate.now())
465                 .horario(HorarioTurno.MANIANA)
466                 .estado(EstadoTurno.PROGRAMADO)
467                 .admin(admin)
468                 .enfermero(enfermero)
469                 .nombreChofer("Chofer Test")
470                 .nombreMedico("Medico Test")
471                 .controlInsumos(new ArrayList<>())
472                 .build();
473         turnoSinControl.setId(20L);
474 
475         when(repository.findById(20L)).thenReturn(
    Optional.of(turnoSinControl));
476         when(ambulanciaRepository.
    findByIdAndEliminadoIsFalse(5L)).thenReturn(Optional
    .of(ambulancia));
477         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(2L)).thenReturn(Optional
    .of(enfermero));
```

```
478         when(repository.save(any(Turno.class))).
    thenReturn(turnoSinControl);
479 
480         TurnoDTO resultado = turnoService.
    modificarTurno(20L, turnoRequestDTO);
481 
482         assertNotNull(resultado);
483         verify(repository, times(1)).findById(20L);
484         verify(ambulanciaRepository, times(1)).
    findByIdAndEliminadoIsFalse(5L);
485         verify(usuarioRepository, times(1)).
    findByIdAndEliminadoIsFalse(2L);
486         verify(repository, times(1)).save(
    turnoSinControl);
487     }
488 
489     @Test
490     void
    modificarTurno_errorCambiarAmbulanciaConControlInsum
    os() {
491         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
492         when(ambulanciaRepository.
    findByIdAndEliminadoIsFalse(5L)).thenReturn(Optional
    .of(ambulancia));
493         when(usuarioRepository.
    findByIdAndEliminadoIsFalse(2L)).thenReturn(Optional
    .of(enfermero));
494 
495         assertThrows(IllegalStateException.class
    , () -> turnoService.modificarTurno(20L,
    turnoRequestDTO));
496         verify(repository, times(1)).findById(20L);
497         verify(repository, never()).save(any(Turno.
    class));
498     }
499 
500     @Test
501     void modificarTurno_ok_sinAmbulanciaNiEnfermero
    () {
502         TurnoRequestDTO requestSinRef = new
```

```
502 TurnoRequestDTO(
503                 LocalDate.now(),
504                 HorarioTurno.TARDE,
505                 null,
506                 null,
507                 null,
508                 "Nuevo Chofer",
509                 "Nuevo Medico"
510         );
511         when(repository.findById(20L)).thenReturn(
    Optional.of(turno));
512         when(repository.save(any(Turno.class))).
    thenReturn(turno);
513 
514         TurnoDTO resultado = turnoService.
    modificarTurno(20L, requestSinRef);
515 
516         assertNotNull(resultado);
517         assertEquals(HorarioTurno.TARDE, turno.
    getHorario());
518         assertEquals("Nuevo Chofer", turno.
    getNombreChofer());
519         verify(repository, times(1)).findById(20L);
520         verify(ambulanciaRepository, never()).
    findByIdAndEliminadoIsFalse(anyLong());
521         verify(usuarioRepository, never()).
    findByIdAndEliminadoIsFalse(anyLong());
522         verify(repository, times(1)).save(turno);
523     }
524 }
525 
```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.dto.InsumoDTO;
import com.tup.tfi.control107.dto.request.
  InsumoRequestDTO;
import com.tup.tfi.control107.model.entity.
  CategoriaInsumo;
import com.tup.tfi.control107.model.entity.Insumo;
import com.tup.tfi.control107.repository.
  CategoriaRepository;
import com.tup.tfi.control107.repository.
  InsumoRepository;
import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class InsumoServiceImplTest {

    @Mock
    private InsumoRepository insumoRepository;

    @Mock
    private CategoriaRepository categoriaRepository;

    @InjectMocks
    private InsumoServiceImpl insumoService;

    private CategoriaInsumo categoria;
    private Insumo insumo;
```

```
38      private InsumoRequestDTO requestDTO;
39
40      private static final String NOMBRE_CATEGORIA = "
    FARMACOLOGICOS";
41      private static final String COLOR_CATEGORIA = "#
    D9574C";
42      private static final String NOMBRE_INSUMO = "
    Adrenalina 1mg";
43
44      @BeforeEach
45      void setUp() {
46          categoria = CategoriaInsumo.builder()
47                  .nombre(NOMBRE_CATEGORIA)
48                  .color(COLOR_CATEGORIA)
49                  .build();
50          categoria.setId(10L);
51
52          insumo = Insumo.builder()
53                  .nombre(NOMBRE_INSUMO)
54                  .categoria(categoria)
55                  .puntoControl(20)
56                  .critico(true)
57                  .build();
58          insumo.setId(1L);
59
60          requestDTO = new InsumoRequestDTO(
    NOMBRE_INSUMO, 10L, 20, true);
61      }
62
63      @Test
64      void listarTodos_ok() {
65          when(insumoRepository.
    findAllByEliminadoIsFalse()).thenReturn(List.of(
    insumo));
66
67          List<InsumoDTO> resultado = insumoService.
    listarTodos();
68
69          assertNotNull(resultado);
70          assertEquals(1, resultado.size());
71          assertEquals(NOMBRE_INSUMO, resultado.get(0).
```

```
 71 getNombre());
 72         verify(insumoRepository, times(1)).
    findAllByEliminadoIsFalse();
 73     }
 74
 75     @Test
 76     void listarCriticos_ok() {
 77         when(insumoRepository.
    findByCriticoIsTrueAndEliminadoIsFalse()).thenReturn
    (List.of(insumo));
 78
 79         List<InsumoDTO> resultado = insumoService.
    listarCriticos();
 80
 81         assertNotNull(resultado);
 82         assertEquals(1, resultado.size());
 83         assertTrue(resultado.get(0).isEsCritico());
 84         verify(insumoRepository, times(1)).
    findByCriticoIsTrueAndEliminadoIsFalse();
 85     }
 86
 87     @Test
 88     void listarPorCategoria_ok() {
 89         when(insumoRepository.
    findByCategoriaIdAndEliminadoIsFalse(10L)).
    thenReturn(List.of(insumo));
 90
 91         List<InsumoDTO> resultado = insumoService.
    listarPorCategoria(10L);
 92
 93         assertNotNull(resultado);
 94         assertEquals(1, resultado.size());
 95         assertEquals(NOMBRE_CATEGORIA, resultado.get
    (0).getCategoria());
 96         verify(insumoRepository, times(1)).
    findByCategoriaIdAndEliminadoIsFalse(10L);
 97     }
 98
 99     @Test
100     void obtenerPorId_ok() {
101         when(insumoRepository.findById(1L)).
```

```
101 thenReturn(Optional.of(insumo));
102
103         InsumoDTO resultado = insumoService.
    obtenerPorId(1L);
104
105         assertNotNull(resultado);
106         assertEquals(1L, resultado.getId());
107         assertEquals(NOMBRE_INSUMO, resultado.
    getNombre());
108         verify(insumoRepository, times(1)).findById(
    1L);
109     }
110
111     @Test
112     void obtenerPorId_errorNoEncontrado() {
113         when(insumoRepository.findById(1L)).
    thenReturn(Optional.empty());
114
115         assertThrows(EntityNotFoundException.class
    , () -> insumoService.obtenerPorId(1L));
116         verify(insumoRepository, times(1)).findById(
    1L);
117     }
118
119     @Test
120     void crearInsumo_ok() {
121         when(categoriaRepository.
    findByIdAndEliminadoIsFalse(10L)).thenReturn(
    Optional.of(categoria));
122         when(insumoRepository.save(any(Insumo.class
    ))).thenReturn(insumo);
123
124         InsumoDTO resultado = insumoService.
    crearInsumo(requestDTO);
125
126         assertNotNull(resultado);
127         assertEquals(NOMBRE_INSUMO, resultado.
    getNombre());
128         verify(categoriaRepository, times(1)).
    findByIdAndEliminadoIsFalse(10L);
129         verify(insumoRepository, times(1)).save(any(
```

```
129 Insumo.class));
130     }
131
132     @Test
133     void crearInsumo_errorCategoriaNoEncontrada() {
134         when(categoriaRepository.
    findByIdAndEliminadoIsFalse(10L)).thenReturn(
    Optional.empty());
135
136         assertThrows(EntityNotFoundException.class
    , () -> insumoService.crearInsumo(requestDTO));
137         verify(categoriaRepository, times(1)).
    findByIdAndEliminadoIsFalse(10L);
138         verify(insumoRepository, never()).save(any(
    Insumo.class));
139     }
140
141     @Test
142     void crearInsumo_errorNombreDuplicado() {
143         when(insumoRepository.
    existsByNombreIgnoreCaseAndEliminadoIsFalse(
    NOMBRE_INSUMO)).thenReturn(true);
144
145         IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class, () -> insumoService.
    crearInsumo(requestDTO));
146
147         assertEquals("Ya existe un insumo con ese
    nombre", ex.getMessage());
148         verify(categoriaRepository, never()).
    findByIdAndEliminadoIsFalse(anyLong());
149         verify(insumoRepository, never()).save(any(
    Insumo.class));
150     }
151
152     @Test
153     void actualizarInsumo_ok_conCategoria() {
154         when(insumoRepository.findById(1L)).
    thenReturn(Optional.of(insumo));
155         when(categoriaRepository.
    findByIdAndEliminadoIsFalse(10L)).thenReturn(
```

```
155 Optional.of(categoria));
156         when(insumoRepository.save(any(Insumo.class
    ))).thenReturn(insumo);
157 
158         InsumoDTO resultado = insumoService.
    actualizarInsumo(1L, requestDTO);
159 
160         assertNotNull(resultado);
161         assertEquals(NOMBRE_INSUMO, resultado.
    getNombre());
162         verify(insumoRepository, times(1)).findById(
    1L);
163         verify(categoriaRepository, times(1)).
    findByIdAndEliminadoIsFalse(10L);
164         verify(insumoRepository, times(1)).save(
    insumo);
165     }
166 
167     @Test
168     void actualizarInsumo_errorNombreDuplicado() {
169         when(insumoRepository.
    existsByNombreIgnoreCaseAndEliminadoIsFalseAndIdNot(
    NOMBRE_INSUMO, 1L)).thenReturn(true);
170 
171         IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class, () -> insumoService.
    actualizarInsumo(1L, requestDTO));
172 
173         assertEquals("Ya existe un insumo con ese
    nombre", ex.getMessage());
174         verify(insumoRepository, never()).findById(
    anyLong());
175         verify(insumoRepository, never()).save(any(
    Insumo.class));
176     }
177 
178     @Test
179     void actualizarInsumo_ok_sinCategoria() {
180         InsumoRequestDTO requestSinCategoria = new
    InsumoRequestDTO(NOMBRE_INSUMO, null, 20, true);
181         when(insumoRepository.findById(1L)).
```

```
181 thenReturn(Optional.of(insumo));
182         when(insumoRepository.save(any(Insumo.class
    ))).thenReturn(insumo);
183 
184         InsumoDTO resultado = insumoService.
    actualizarInsumo(1L, requestSinCategoria);
185 
186         assertNotNull(resultado);
187         verify(insumoRepository, times(1)).findById(
    1L);
188         verify(categoriaRepository, never()).
    findByIdAndEliminadoIsFalse(anyLong());
189         verify(insumoRepository, times(1)).save(
    insumo);
190     }
191 
192     @Test
193     void actualizarInsumo_errorInsumoNoEncontrado
    () {
194         when(insumoRepository.findById(1L)).
    thenReturn(Optional.empty());
195 
196         assertThrows(EntityNotFoundException.class
    , () -> insumoService.actualizarInsumo(1L,
    requestDTO));
197         verify(insumoRepository, times(1)).findById(
    1L);
198         verify(insumoRepository, never()).save(any(
    Insumo.class));
199     }
200 
201     @Test
202     void actualizarInsumo_errorCategoriaNoEncontrada
    () {
203         when(insumoRepository.findById(1L)).
    thenReturn(Optional.of(insumo));
204         when(categoriaRepository.
    findByIdAndEliminadoIsFalse(10L)).thenReturn(
    Optional.empty());
205 
206         assertThrows(EntityNotFoundException.class
```

```
206 , () -> insumoService.actualizarInsumo(1L,
    requestDTO));
207         verify(insumoRepository, times(1)).findById(
    1L);
208         verify(categoriaRepository, times(1)).
    findByIdAndEliminadoIsFalse(10L);
209         verify(insumoRepository, never()).save(any(
    Insumo.class));
210     }
211
212     @Test
213     void eliminarPorId_ok() {
214         when(insumoRepository.findById(1L)).
    thenReturn(Optional.of(insumo));
215
216         insumoService.eliminarPorId(1L);
217
218         assertTrue(insumo.isEliminado());
219         verify(insumoRepository, times(1)).findById(
    1L);
220         verify(insumoRepository, times(1)).save(
    insumo);
221     }
222
223     @Test
224     void eliminarPorId_errorNoEncontrado() {
225         when(insumoRepository.findById(1L)).
    thenReturn(Optional.empty());
226
227         assertThrows(EntityNotFoundException.class
    , () -> insumoService.eliminarPorId(1L));
228         verify(insumoRepository, times(1)).findById(
    1L);
229         verify(insumoRepository, never()).save(any(
    Insumo.class));
230     }
231 }
232
```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.dto.UsuarioDTO;
import com.tup.tfi.control107.dto.request.
UsuarioRequestDTO;
import com.tup.tfi.control107.model.entity.Usuario;
import com.tup.tfi.control107.model.enums.RolUsuario;
import com.tup.tfi.control107.repository.
UsuarioRepository;
import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class UsuarioServiceImplTest {

    @Mock
    private UsuarioRepository repository;

    @InjectMocks
    private UsuarioServiceImpl usuarioService;

    private Usuario usuario;
    private UsuarioRequestDTO requestDTO;

    private static final String MAIL_ADMIN = "hugo.
catalan@control107.gob.ar";
    private static final String DNI_ADMIN = "30111222
";
    private static final String NOMBRE_ADMIN = "Hugo
```

```
37 Catalan";
38
39     @BeforeEach
40     void setUp() {
41         usuario = Usuario.builder()
42                 .nombre(NOMBRE_ADMIN)
43                 .dni(DNI_ADMIN)
44                 .email(MAIL_ADMIN)
45                 .rol(RolUsuario.ADMINISTRADOR)
46                 .activo(true)
47                 .build();
48         usuario.setId(1L);
49
50         requestDTO = new UsuarioRequestDTO(
   NOMBRE_ADMIN, DNI_ADMIN, MAIL_ADMIN, RolUsuario.
   ADMINISTRADOR, "1234",true);
51     }
52
53     @Test
54     void listarTodos_ok() {
55         when(repository.findAllByEliminadoIsFalse()).
   thenReturn(List.of(usuario));
56
57         List<UsuarioDTO> resultado = usuarioService.
   listarTodos();
58
59         assertNotNull(resultado);
60         assertEquals(1, resultado.size());
61         assertEquals(NOMBRE_ADMIN, resultado.get(0).
   getNombre());
62         verify(repository, times(1)).
   findAllByEliminadoIsFalse();
63     }
64
65     @Test
66     void obtenerPorId_ok() {
67         when(repository.findById(1L)).thenReturn(
   Optional.of(usuario));
68
69         UsuarioDTO resultado = usuarioService.
   obtenerPorId(1L);
```

```
 70
 71          assertNotNull(resultado);
 72          assertEquals(1L, resultado.getId());
 73          assertEquals(NOMBRE_ADMIN, resultado.
    getNombre());
 74          verify(repository, times(1)).findById(1L);
 75      }
 76
 77      @Test
 78      void obtenerPorId_errorNoEncontrado() {
 79          when(repository.findById(1L)).thenReturn(
    Optional.empty());
 80
 81          assertThrows(EntityNotFoundException.class
    , () -> usuarioService.obtenerPorId(1L));
 82          verify(repository, times(1)).findById(1L);
 83      }
 84
 85      @Test
 86      void crearUsuario_ok() {
 87          when(repository.save(any(Usuario.class))).
    thenReturn(usuario);
 88
 89          UsuarioDTO resultado = usuarioService.
    crearUsuario(requestDTO);
 90
 91          assertNotNull(resultado);
 92          assertEquals(NOMBRE_ADMIN, resultado.
    getNombre());
 93          verify(repository, times(1)).save(any(
    Usuario.class));
 94      }
 95
 96      @Test
 97      void crearUsuario_errorDniDuplicado() {
 98          when(repository.
    existsByDniAndEliminadoIsFalse(DNI_ADMIN)).
    thenReturn(true);
 99
100          IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class, () -> usuarioService
```

```
100 .crearUsuario(requestDTO));
101
102         assertEquals("Ya existe un usuario con ese
    DNI", ex.getMessage());
103         verify(repository, never()).save(any(Usuario
    .class));
104     }
105
106     @Test
107     void crearUsuario_errorEmailDuplicado() {
108         when(repository.
    existsByDniAndEliminadoIsFalse(DNI_ADMIN)).
    thenReturn(false);
109         when(repository.
    existsByEmailIgnoreCaseAndEliminadoIsFalse(
    MAIL_ADMIN)).thenReturn(true);
110
111         IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class, () -> usuarioService
    .crearUsuario(requestDTO));
112
113         assertEquals("Ya existe un usuario con ese
    email", ex.getMessage());
114         verify(repository, never()).save(any(Usuario
    .class));
115     }
116
117     @Test
118     void actualizarUsuario_ok() {
119         when(repository.findById(1L)).thenReturn(
    Optional.of(usuario));
120         when(repository.save(any(Usuario.class))).
    thenReturn(usuario);
121
122         UsuarioDTO resultado = usuarioService.
    actualizarUsuario(1L, requestDTO);
123
124         assertNotNull(resultado);
125         assertEquals(NOMBRE_ADMIN, resultado.
    getNombre());
126         verify(repository, times(1)).findById(1L);
```

```
127         verify(repository, times(1)).save(usuario);
128     }
129 
130     @Test
131     void actualizarUsuario_errorDniDuplicado() {
132         when(repository.
    existsByDniAndEliminadoIsFalseAndIdNot(DNI_ADMIN, 1L
    )).thenReturn(true);
133 
134         IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class, () -> usuarioService
    .actualizarUsuario(1L, requestDTO));
135 
136         assertEquals("Ya existe un usuario con ese
    DNI", ex.getMessage());
137         verify(repository, never()).findById(anyLong
    ());
138         verify(repository, never()).save(any(Usuario
    .class));
139     }
140 
141     @Test
142     void actualizarUsuario_errorEmailDuplicado() {
143         when(repository.
    existsByDniAndEliminadoIsFalseAndIdNot(DNI_ADMIN, 1L
    )).thenReturn(false);
144         when(repository.
    existsByEmailIgnoreCaseAndEliminadoIsFalseAndIdNot(
    MAIL_ADMIN, 1L)).thenReturn(true);
145 
146         IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class, () -> usuarioService
    .actualizarUsuario(1L, requestDTO));
147 
148         assertEquals("Ya existe un usuario con ese
    email", ex.getMessage());
149         verify(repository, never()).findById(anyLong
    ());
150         verify(repository, never()).save(any(Usuario
    .class));
151     }
```

```
152
153     @Test
154     void actualizarUsuario_errorNoEncontrado() {
155         when(repository.findById(1L)).thenReturn(
    Optional.empty());
156
157         assertThrows(EntityNotFoundException.class
    , () -> usuarioService.actualizarUsuario(1L,
    requestDTO));
158         verify(repository, times(1)).findById(1L);
159         verify(repository, never()).save(any(Usuario
    .class));
160     }
161
162     @Test
163     void eliminarPorId_ok() {
164         when(repository.findById(1L)).thenReturn(
    Optional.of(usuario));
165
166         usuarioService.eliminarPorId(1L);
167
168         assertTrue(usuario.isEliminado());
169         verify(repository, times(1)).findById(1L);
170         verify(repository, times(1)).save(usuario);
171     }
172
173     @Test
174     void eliminarPorId_errorNoEncontrado() {
175         when(repository.findById(1L)).thenReturn(
    Optional.empty());
176
177         assertThrows(EntityNotFoundException.class
    , () -> usuarioService.eliminarPorId(1L));
178         verify(repository, times(1)).findById(1L);
179         verify(repository, never()).save(any(Usuario
    .class));
180     }
181 }
182
```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.dto.AmbulanciaDTO;
import com.tup.tfi.control107.dto.request.
AmbulanciaRequestDTO;
import com.tup.tfi.control107.model.entity.Ambulancia
;
import com.tup.tfi.control107.model.entity.Insumo;
import com.tup.tfi.control107.repository.
AmbulanciaRepository;
import com.tup.tfi.control107.repository.
InsumoRepository;
import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.List;
import java.util.Optional;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class AmbulanciaServiceImplTest {

    @Mock
    private AmbulanciaRepository ambulanciaRepository
;

    @Mock
    private InsumoRepository insumoRepository;

    @InjectMocks
    private AmbulanciaServiceImpl
ambulanciaServiceImpl;
```

```
36
37      private Ambulancia ambulancia;
38      private Insumo insumo;
39      private AmbulanciaRequestDTO requestDTO;
40
41      private static final String PATENTE = "AB123CD";
42
43      @BeforeEach
44      void setUp() {
45          insumo = Insumo.builder()
46                  .nombre("Gasas")
47                  .puntoControl(10)
48                  .critico(false)
49                  .build();
50          insumo.setId(100L);
51
52          ambulancia = Ambulancia.builder()
53                  .patente(PATENTE)
54                  .disponible(true)
55                  .insumos(Set.of(insumo))
56                  .build();
57          ambulancia.setId(1L);
58
59          requestDTO = new AmbulanciaRequestDTO(PATENTE
    , true, Set.of(100L));
60      }
61
62      @Test
63      void listarTodas_ok() {
64          when(ambulanciaRepository.
    findAllByEliminadoIsFalse()).thenReturn(List.of(
    ambulancia));
65
66          List<AmbulanciaDTO> resultado =
    ambulanciaServiceImpl.listarTodas();
67
68          assertNotNull(resultado);
69          assertEquals(1, resultado.size());
70          assertEquals(PATENTE, resultado.get(0).
    getPatente());
71          verify(ambulanciaRepository, times(1)).
```

```
 71 findAllByEliminadoIsFalse();
 72     }
 73
 74     @Test
 75     void obtenerPorId_ok() {
 76         when(ambulanciaRepository.findById(1L)).
    thenReturn(Optional.of(ambulancia));
 77
 78         AmbulanciaDTO resultado =
    ambulanciaServiceImpl.obtenerPorId(1L);
 79
 80         assertNotNull(resultado);
 81         assertEquals(1L, resultado.getId());
 82         assertEquals(PATENTE, resultado.getPatente
    ());
 83         verify(ambulanciaRepository, times(1)).
    findById(1L);
 84     }
 85
 86     @Test
 87     void obtenerPorId_errorNoEncontrada() {
 88         when(ambulanciaRepository.findById(1L)).
    thenReturn(Optional.empty());
 89
 90         assertThrows(EntityNotFoundException.class
    , () -> ambulanciaServiceImpl.obtenerPorId(1L));
 91         verify(ambulanciaRepository, times(1)).
    findById(1L);
 92     }
 93
 94     @Test
 95     void crearAmbulancia_ok() {
 96         when(insumoRepository.
    findByIdAndEliminadoIsFalse(100L)).thenReturn(
    Optional.of(insumo));
 97         when(ambulanciaRepository.save(any(
    Ambulancia.class))).thenReturn(ambulancia);
 98
 99         AmbulanciaDTO resultado =
    ambulanciaServiceImpl.crearAmbulancia(requestDTO);
100
```

```
101         assertNotNull(resultado);
102         assertEquals(PATENTE, resultado.getPatente
    ());
103         verify(insumoRepository, times(1)).
    findByIdAndEliminadoIsFalse(100L);
104         verify(ambulanciaRepository, times(1)).save(
    any(Ambulancia.class));
105     }
106 
107     @Test
108     void crearAmbulancia_errorInsumoNoEncontrado() {
109         when(insumoRepository.
    findByIdAndEliminadoIsFalse(100L)).thenReturn(
    Optional.empty());
110 
111         assertThrows(EntityNotFoundException.class
    , () -> ambulanciaServiceImpl.crearAmbulancia(
    requestDTO));
112         verify(insumoRepository, times(1)).
    findByIdAndEliminadoIsFalse(100L);
113         verify(ambulanciaRepository, never()).save(
    any(Ambulancia.class));
114     }
115 
116     @Test
117     void crearAmbulancia_errorPatenteDuplicada() {
118         when(ambulanciaRepository.
    existsByPatenteIgnoreCaseAndEliminadoIsFalse(PATENTE
    )).thenReturn(true);
119 
120         IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class, () ->
    ambulanciaServiceImpl.crearAmbulancia(requestDTO));
121 
122         assertEquals("Ya existe una ambulancia con
    esa patente", ex.getMessage());
123         verify(insumoRepository, never()).
    findByIdAndEliminadoIsFalse(anyLong());
124         verify(ambulanciaRepository, never()).save(
    any(Ambulancia.class));
125     }
```

```
126
127     @Test
128     void actualizarAmbulancia() {
129         when(ambulanciaRepository.findById(1L)).
    thenReturn(Optional.of(ambulancia));
130         when(insumoRepository.
    findByIdAndEliminadoIsFalse(100L)).thenReturn(
    Optional.of(insumo));
131         when(ambulanciaRepository.save(any(
    Ambulancia.class))).thenReturn(ambulancia);
132
133         AmbulanciaDTO resultado =
    ambulanciaServiceImpl.actualizarAmbulancia(1L,
    requestDTO);
134
135         assertNotNull(resultado);
136         assertEquals(PATENTE, resultado.getPatente
    ());
137         verify(ambulanciaRepository, times(1)).
    findById(1L);
138         verify(insumoRepository, times(1)).
    findByIdAndEliminadoIsFalse(100L);
139         verify(ambulanciaRepository, times(1)).save(
    ambulancia);
140     }
141
142     @Test
143     void actualizarAmbulancia_errorPatenteDuplicada
    () {
144         when(ambulanciaRepository.
    existsByPatenteIgnoreCaseAndEliminadoIsFalseAndIdNot
    (PATENTE, 1L)).thenReturn(true);
145
146         IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class, () ->
    ambulanciaServiceImpl.actualizarAmbulancia(1L,
    requestDTO));
147
148         assertEquals("Ya existe una ambulancia con
    esa patente", ex.getMessage());
149         verify(ambulanciaRepository, never()).
```

```
149 findById(anyLong());
150         verify(ambulanciaRepository, never()).save(
    any(Ambulancia.class));
151     }
152 
153     @Test
154     void
    actualizarAmbulancia_errorInsumoNoEncontrado() {
155         when(ambulanciaRepository.findById(1L)).
    thenReturn(Optional.of(ambulancia));
156         when(insumoRepository.
    findByIdAndEliminadoIsFalse(100L)).thenReturn(
    Optional.empty());
157 
158         assertThrows(EntityNotFoundException.class
    , () -> ambulanciaServiceImpl.actualizarAmbulancia(
    1L, requestDTO));
159         verify(ambulanciaRepository, times(1)).
    findById(1L);
160         verify(insumoRepository, times(1)).
    findByIdAndEliminadoIsFalse(100L);
161         verify(ambulanciaRepository, never()).save(
    any(Ambulancia.class));
162     }
163 
164     @Test
165     void eliminarPorId_ok() {
166         when(ambulanciaRepository.findById(1L)).
    thenReturn(Optional.of(ambulancia));
167 
168         ambulanciaServiceImpl.eliminarPorId(1L);
169 
170         assertTrue(ambulancia.isEliminado());
171         verify(ambulanciaRepository, times(1)).
    findById(1L);
172         verify(ambulanciaRepository, times(1)).save(
    ambulancia);
173     }
174 
175     @Test
176     void eliminarPorId_errorNoEncontrada() {
```

```
177         when(ambulanciaRepository.findById(1L)).
    thenReturn(Optional.empty());
178 
179         assertThrows(EntityNotFoundException.class
    , () -> ambulanciaServiceImpl.eliminarPorId(1L));
180         verify(ambulanciaRepository, times(1)).
    findById(1L);
181         verify(ambulanciaRepository, never()).save(
    any(Ambulancia.class));
182     }
183 }
```

```java
package com.tup.tfi.control107.service.impl;

import com.tup.tfi.control107.dto.CategoriaInsumoDTO;
import com.tup.tfi.control107.dto.request.
  CategoriaInsumoRequestDTO;
import com.tup.tfi.control107.model.entity.
  CategoriaInsumo;
import com.tup.tfi.control107.repository.
  CategoriaRepository;
import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class CategoriaInsumoServiceImplTest {

    @Mock
    private CategoriaRepository repository;

    @InjectMocks
    private CategoriaInsumoServiceImpl
  categoriaService;

    private CategoriaInsumo categoria;
    private CategoriaInsumoRequestDTO requestDTO;

    private static final String NOMBRE_VALIDO = "
  FARMACOLOGICOS";
    private static final String COLOR_VALIDO = "#
  D9574C";
```

```
36
37      @BeforeEach
38      void setUp() {
39          categoria = CategoriaInsumo.builder()
40                  .nombre(NOMBRE_VALIDO)
41                  .color(COLOR_VALIDO)
42                  .build();
43          categoria.setId(1L);
44
45          requestDTO = new CategoriaInsumoRequestDTO(
    NOMBRE_VALIDO, COLOR_VALIDO);
46      }
47
48      @Test
49      void listarTodas_ok() {
50          when(repository.findAllByEliminadoIsFalse()).
    thenReturn(List.of(categoria));
51
52          List<CategoriaInsumoDTO> resultado =
    categoriaService.listarTodas();
53
54          assertNotNull(resultado);
55          assertEquals(1, resultado.size());
56          assertEquals(NOMBRE_VALIDO, resultado.get(0).
    getNombre());
57          verify(repository, times(1)).
    findAllByEliminadoIsFalse();
58      }
59
60      @Test
61      void obtenerPorId_ok() {
62          when(repository.findById(1L)).thenReturn(
    Optional.of(categoria));
63
64          CategoriaInsumoDTO resultado =
    categoriaService.obtenerPorId(1L);
65
66          assertNotNull(resultado);
67          assertEquals(1L, resultado.getId());
68          assertEquals(NOMBRE_VALIDO, resultado.
    getNombre());
```

```
69          verify(repository, times(1)).findById(1L);
70      }
71
72      @Test
73      void obtenerPorId_errorNoEncontrada() {
74          when(repository.findById(1L)).thenReturn(
    Optional.empty());
75
76          assertThrows(EntityNotFoundException.class
    , () -> categoriaService.obtenerPorId(1L));
77          verify(repository, times(1)).findById(1L);
78      }
79
80      @Test
81      void crearCategoria_ok() {
82          when(repository.save(any(CategoriaInsumo.
    class))).thenReturn(categoria);
83
84          CategoriaInsumoDTO resultado =
    categoriaService.crearCategoria(requestDTO);
85
86          assertNotNull(resultado);
87          assertEquals(NOMBRE_VALIDO, resultado.
    getNombre());
88          verify(repository, times(1)).save(any(
    CategoriaInsumo.class));
89      }
90
91      @Test
92      void crearCategoria_errorNombreDuplicado() {
93          when(repository.
    existsByNombreIgnoreCaseAndEliminadoIsFalse(
    NOMBRE_VALIDO)).thenReturn(true);
94
95          IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class, () ->
    categoriaService.crearCategoria(requestDTO));
96
97          assertEquals("Ya existe una categoría con
    ese nombre", ex.getMessage());
98          verify(repository, never()).save(any(
```

```
 98 CategoriaInsumo.class));
 99     }
100 
101     @Test
102     void actualizarCategoria_ok() {
103         when(repository.findById(1L)).thenReturn(
    Optional.of(categoria));
104         when(repository.save(any(CategoriaInsumo.
    class))).thenReturn(categoria);
105 
106         CategoriaInsumoDTO resultado =
    categoriaService.actualizarCategoria(1L, requestDTO
    );
107 
108         assertNotNull(resultado);
109         assertEquals(NOMBRE_VALIDO, resultado.
    getNombre());
110         verify(repository, times(1)).findById(1L);
111         verify(repository, times(1)).save(categoria
    );
112     }
113 
114     @Test
115     void actualizarCategoria_errorNombreDuplicado
    () {
116         when(repository.
    existsByNombreIgnoreCaseAndEliminadoIsFalseAndIdNot(
    NOMBRE_VALIDO, 1L)).thenReturn(true);
117 
118         IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class, () ->
    categoriaService.actualizarCategoria(1L, requestDTO
    ));
119 
120         assertEquals("Ya existe una categoría con
    ese nombre", ex.getMessage());
121         verify(repository, never()).save(any(
    CategoriaInsumo.class));
122     }
123 
124     @Test
```

```
125     void actualizarCategoria_errorNoEncontrada() {
126         when(repository.findById(1L)).thenReturn(
    Optional.empty());
127 
128         assertThrows(EntityNotFoundException.class
    , () -> categoriaService.actualizarCategoria(1L,
    requestDTO));
129         verify(repository, times(1)).findById(1L);
130         verify(repository, never()).save(any(
    CategoriaInsumo.class));
131     }
132 
133     @Test
134     void eliminarPorId_ok() {
135         when(repository.findById(1L)).thenReturn(
    Optional.of(categoria));
136 
137         categoriaService.eliminarPorId(1L);
138 
139         assertTrue(categoria.isEliminado());
140         verify(repository, times(1)).findById(1L);
141         verify(repository, times(1)).save(categoria
    );
142     }
143 
144     @Test
145     void eliminarPorId_errorNoEncontrada() {
146         when(repository.findById(1L)).thenReturn(
    Optional.empty());
147 
148         assertThrows(EntityNotFoundException.class
    , () -> categoriaService.eliminarPorId(1L));
149         verify(repository, times(1)).findById(1L);
150         verify(repository, never()).save(any(
    CategoriaInsumo.class));
151     }
152 }
153 
```

```java
package com.tup.tfi.control107.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.tup.tfi.control107.config.SecurityConfig;
import com.tup.tfi.control107.dto.LoginDTO;
import com.tup.tfi.control107.dto.request.
LoginRequestDTO;
import com.tup.tfi.control107.exception.
CredencialesInvalidasException;
import com.tup.tfi.control107.service.AuthService;
import com.tup.tfi.control107.util.JwtUtil;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.
Autowired;
import org.springframework.boot.webmvc.test.
autoconfigure.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.security.test.context.
support.WithMockUser;
import org.springframework.test.context.bean.override
.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.
request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.
result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.
result.MockMvcResultMatchers.status;

@WebMvcTest(AuthController.class)
@Import(SecurityConfig.class)
@WithMockUser
class AuthControllerTest {


    @Autowired
    private MockMvc mockMvc;
```

```
33
34      private final ObjectMapper objectMapper = new
    ObjectMapper();
35
36      @MockitoBean
37      private AuthService authService;
38
39      @MockitoBean
40      private JwtUtil jwtUtil;
41
42      private static final String CONTEXT_PATH = "/api"
    ;
43      private static final String BASE_URL = "/api/v1/
    auth";
44
45      @Test
46      void login_ok() throws Exception {
47          LoginRequestDTO request = new LoginRequestDTO
    ("admin", "admin123");
48          LoginDTO response = new LoginDTO("mock-token"
    , 3600L, "Bearer", "message:read");
49
50          when(authService.login(any(LoginRequestDTO.
    class))).thenReturn(response);
51
52          mockMvc.perform(post(BASE_URL + "/login")
53                          .contextPath(CONTEXT_PATH)
54                          .contentType(MediaType.
    APPLICATION_JSON)
55                          .content(objectMapper.
    writeValueAsString(request)))
56                  .andExpect(status().isOk())
57                  .andExpect(jsonPath("$.access_token"
    ).value("mock-token"))
58                  .andExpect(jsonPath("$.token_type").
    value("Bearer"))
59                  .andExpect(jsonPath("$.scope").value(
    "message:read"));
60      }
61
62      @Test
```

```
63      void login_credencialesInvalidas_unauthorized() throws Exception {
64          LoginRequestDTO request = new LoginRequestDTO("admin", "wrong-pwd");
65
66          when(authService.login(any(LoginRequestDTO.class)))
67                  .thenThrow(new CredencialesInvalidasException());
68
69          mockMvc.perform(post(BASE_URL + "/login")
70                          .contextPath(CONTEXT_PATH)
71                          .contentType(MediaType.APPLICATION_JSON)
72                          .content(objectMapper.writeValueAsString(request)))
73                  .andExpect(status().isUnauthorized())
74                  .andExpect(jsonPath("$.status").value(401))
75                  .andExpect(jsonPath("$.error").value("Credenciales inválidas"));
76      }
77  }
78
```

```java
package com.tup.tfi.control107.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import com.tup.tfi.control107.config.SecurityConfig;
import com.tup.tfi.control107.dto.ResumenOperativoDTO;
import com.tup.tfi.control107.dto.TurnoDTO;
import com.tup.tfi.control107.dto.request.CancelarTurnoDTO;
import com.tup.tfi.control107.dto.request.IngresoEgresoDTO;
import com.tup.tfi.control107.dto.request.TurnoRequestDTO;
import com.tup.tfi.control107.model.enums.EstadoTurno;
import com.tup.tfi.control107.model.enums.HorarioTurno;
import com.tup.tfi.control107.service.TurnoService;
import com.tup.tfi.control107.util.JwtUtil;
import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.time.LocalDate;
import java.util.List;
import java.util.Set;

import static org.hamcrest.Matchers.hasSize;
```

```
31 import static org.mockito.ArgumentMatchers.any;
32 import static org.mockito.ArgumentMatchers.eq;
33 import static org.mockito.Mockito.doNothing;
34 import static org.mockito.Mockito.when;
35 import static org.springframework.test.web.servlet.
   request.MockMvcRequestBuilders.*;
36 import static org.springframework.test.web.servlet.
   result.MockMvcResultMatchers.*;
37
38 @WebMvcTest(TurnoController.class)
39 @Import(SecurityConfig.class)
40 @WithMockUser(roles = "ADMINISTRADOR")
41 class TurnoControllerTest {
42
43
44     @Autowired
45     private MockMvc mockMvc;
46
47     private final ObjectMapper objectMapper = new
   ObjectMapper().registerModule(new JavaTimeModule());
48
49     @MockitoBean
50     private TurnoService turnoService;
51
52     @MockitoBean
53     private JwtUtil jwtUtil;
54
55     private TurnoDTO turnoDTO;
56     private static final Long ID = 1L;
57     private static final String CONTEXT_PATH = "/api"
   ;
58     private static final String BASE_URL = "/api/v1/
   turnos";
59
60     @BeforeEach
61     void setUp() {
62         turnoDTO = TurnoDTO.builder()
63                 .id(ID)
64                 .fecha(LocalDate.of(2026, 10, 15))
65                 .horario(HorarioTurno.MANIANA)
66                 .estado(EstadoTurno.PROGRAMADO)
```

```
67                  .medico("Dr. Perez")
68                  .chofer("Carlos Chofer")
69                  .controlInsumos(Set.of())
70                  .build();
71      }
72
73      @Test
74      void mostrarTurnos_ok() throws Exception {
75          when(turnoService.obtenerTodos()).thenReturn
    (List.of(turnoDTO));
76
77          mockMvc.perform(get(BASE_URL).contextPath(
    CONTEXT_PATH))
78                  .andExpect(status().isOk())
79                  .andExpect(jsonPath("$", hasSize(1
    )))
80                  .andExpect(jsonPath("$[0].id").value
    (ID))
81                  .andExpect(jsonPath("$[0].medico").
    value("Dr. Perez"));
82      }
83
84      @Test
85      void obtenerTurnosActivos_ok() throws Exception
     {
86          when(turnoService.obtenerTurnosActivos()).
    thenReturn(List.of(turnoDTO));
87
88          mockMvc.perform(get(BASE_URL + "/activos").
    contextPath(CONTEXT_PATH))
89                  .andExpect(status().isOk())
90                  .andExpect(jsonPath("$", hasSize(1
    )))
91                  .andExpect(jsonPath("$[0].id").value
    (ID));
92      }
93
94      @Test
95      void obtenerPorId_ok() throws Exception {
96          when(turnoService.obtenerPorId(ID)).
    thenReturn(turnoDTO);
```

```
 97
 98          mockMvc.perform(get(BASE_URL + "/{id}", ID).
    contextPath(CONTEXT_PATH))
 99                  .andExpect(status().isOk())
100                  .andExpect(jsonPath("$.id").value(ID
    ))
101                  .andExpect(jsonPath("$.medico").
    value("Dr. Perez"));
102      }
103
104      @Test
105      void obtenerPorId_notFound() throws Exception {
106          when(turnoService.obtenerPorId(99L))
107                  .thenThrow(new
    EntityNotFoundException("No se encontro Turno con id
     99"));
108
109          mockMvc.perform(get(BASE_URL + "/{id}", 99L
    ).contextPath(CONTEXT_PATH))
110                  .andExpect(status().isNotFound())
111                  .andExpect(jsonPath("$.status").
    value(404));
112      }
113
114      @Test
115      void obtenerResumenOperativo_ok() throws
    Exception {
116          ResumenOperativoDTO resumen =
    ResumenOperativoDTO.builder()
117                  .turnosActivos(2)
118                  .ambulanciasDisponibles(3)
119                  .ambulanciasTotales(5)
120                  .alertas(1)
121                  .turnosConInconsistencias(List.of(
    turnoDTO))
122                  .listaAmbulancias(List.of())
123                  .build();
124
125          when(turnoService.obtenerResumenOperativo
    ()).thenReturn(resumen);
126
```

```
127         mockMvc.perform(get(BASE_URL + "/resumen").
    contextPath(CONTEXT_PATH))
128                 .andExpect(status().isOk())
129                 .andExpect(jsonPath("$.turnosActivos
    ").value(2))
130                 .andExpect(jsonPath("$.
    ambulanciasDisponibles").value(3));
131     }
132 
133     @Test
134     void crearTurno_ok() throws Exception {
135         TurnoRequestDTO request = new
    TurnoRequestDTO(
136                 LocalDate.of(2026, 10, 15),
137                 HorarioTurno.MANIANA,
138                 1L, 2L, 3L,
139                 "Carlos Chofer", "Dr. Perez"
140         );
141 
142         when(turnoService.crearTurno(any(
    TurnoRequestDTO.class))).thenReturn(turnoDTO);
143 
144         mockMvc.perform(post(BASE_URL)
145                         .contextPath(CONTEXT_PATH)
146                         .contentType(MediaType.
    APPLICATION_JSON)
147                         .content(objectMapper.
    writeValueAsString(request)))
148                 .andExpect(status().isOk())
149                 .andExpect(jsonPath("$.id").value(ID
    ));
150     }
151 
152     @Test
153     void modificarTurno_ok() throws Exception {
154         TurnoRequestDTO request = new
    TurnoRequestDTO(
155                 LocalDate.of(2026, 10, 15),
156                 HorarioTurno.MANIANA,
157                 1L, 2L, 3L,
158                 "Carlos Chofer", "Dr. Perez"
```

```
159         );
160 
161         when(turnoService.modificarTurno(eq(ID), any
    (TurnoRequestDTO.class))).thenReturn(turnoDTO);
162 
163         mockMvc.perform(put(BASE_URL + "/{id}", ID)
164                         .contextPath(CONTEXT_PATH)
165                         .contentType(MediaType.
    APPLICATION_JSON)
166                         .content(objectMapper.
    writeValueAsString(request)))
167                 .andExpect(status().isOk())
168                 .andExpect(jsonPath("$.id").value(ID
    ));
169     }
170 
171     @Test
172     void registrarIngreso_ok() throws Exception {
173         IngresoEgresoDTO request = new
    IngresoEgresoDTO(ID, List.of(), "Ingreso OK");
174         doNothing().when(turnoService).
    registrarIngreso(any(IngresoEgresoDTO.class));
175 
176         mockMvc.perform(post(BASE_URL + "/ingreso")
177                         .contextPath(CONTEXT_PATH)
178                         .contentType(MediaType.
    APPLICATION_JSON)
179                         .content(objectMapper.
    writeValueAsString(request)))
180                 .andExpect(status().isOk());
181     }
182 
183     @Test
184     void registrarEgreso_ok() throws Exception {
185         IngresoEgresoDTO request = new
    IngresoEgresoDTO(ID, List.of(), "Egreso OK");
186         doNothing().when(turnoService).
    registrarEgreso(any(IngresoEgresoDTO.class));
187 
188         mockMvc.perform(post(BASE_URL + "/egreso")
189                         .contextPath(CONTEXT_PATH)
```

```
190                         .contentType(MediaType.
    APPLICATION_JSON)
191                         .content(objectMapper.
    writeValueAsString(request)))
192                 .andExpect(status().isOk());
193     }
194
195     @Test
196     void cancelarTurno_ok() throws Exception {
197         CancelarTurnoDTO request = new
    CancelarTurnoDTO("Cancelado por lluvia");
198         TurnoDTO cancelado = TurnoDTO.builder()
199                 .id(ID)
200                 .estado(EstadoTurno.CANCELADO)
201                 .build();
202
203         when(turnoService.cancelarTurno(eq(ID), eq("
    Cancelado por lluvia"))).thenReturn(cancelado);
204
205         mockMvc.perform(post(BASE_URL + "/{id}/
    cancelar", ID)
206                         .contextPath(CONTEXT_PATH)
207                         .contentType(MediaType.
    APPLICATION_JSON)
208                         .content(objectMapper.
    writeValueAsString(request)))
209                 .andExpect(status().isOk())
210                 .andExpect(jsonPath("$.id").value(ID
    ))
211                 .andExpect(jsonPath("$.estado").
    value("CANCELADO"));
212     }
213 }
214
```

```java
package com.tup.tfi.control107.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.tup.tfi.control107.config.SecurityConfig;
import com.tup.tfi.control107.dto.InsumoDTO;
import com.tup.tfi.control107.dto.request.
InsumoRequestDTO;
import com.tup.tfi.control107.service.InsumoService;
import com.tup.tfi.control107.util.JwtUtil;
import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.
Autowired;
import org.springframework.boot.webmvc.test.
autoconfigure.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.security.test.context.
support.WithMockUser;
import org.springframework.test.context.bean.override
.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.List;

import static org.hamcrest.Matchers.hasSize;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.doNothing;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.
request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.
result.MockMvcResultMatchers.*;

@WebMvcTest(InsumoController.class)
@Import(SecurityConfig.class)
@WithMockUser(roles = "ADMINISTRADOR")
class InsumoControllerTest {
```

```
35
36
37      @Autowired
38      private MockMvc mockMvc;
39
40      private final ObjectMapper objectMapper = new
    ObjectMapper();
41
42      @MockitoBean
43      private InsumoService insumoService;
44
45      @MockitoBean
46      private JwtUtil jwtUtil;
47
48      private InsumoDTO insumoDTO;
49      private static final Long ID = 1L;
50      private static final String NOMBRE = "Adrenalina
    1mg";
51      private static final String CATEGORIA = "
    FARMACOLOGICOS";
52      private static final String CONTEXT_PATH = "/api"
    ;
53      private static final String BASE_URL = "/api/v1/
    insumos";
54
55      @BeforeEach
56      void setUp() {
57          insumoDTO = InsumoDTO.builder()
58                  .id(ID)
59                  .nombre(NOMBRE)
60                  .categoria(CATEGORIA)
61                  .puntoControl(20)
62                  .esCritico(true)
63                  .build();
64      }
65
66      @Test
67      void listarTodos_ok() throws Exception {
68          when(insumoService.listarTodos()).thenReturn(
    List.of(insumoDTO));
69
```

```
70          mockMvc.perform(get(BASE_URL).contextPath(
    CONTEXT_PATH))
71                  .andExpect(status().isOk())
72                  .andExpect(jsonPath("$", hasSize(1
    )))
73                  .andExpect(jsonPath("$[0].id").value
    (ID))
74                  .andExpect(jsonPath("$[0].nombre").
    value(NOMBRE))
75                  .andExpect(jsonPath("$[0].categoria"
    ).value(CATEGORIA));
76      }
77
78      @Test
79      void listarCriticos_ok() throws Exception {
80          when(insumoService.listarCriticos()).
    thenReturn(List.of(insumoDTO));
81
82          mockMvc.perform(get(BASE_URL + "/criticos").
    contextPath(CONTEXT_PATH))
83                  .andExpect(status().isOk())
84                  .andExpect(jsonPath("$", hasSize(1
    )))
85                  .andExpect(jsonPath("$[0].esCritico"
    ).value(true));
86      }
87
88      @Test
89      void listarPorCategoria_ok() throws Exception {
90          when(insumoService.listarPorCategoria(10L)).
    thenReturn(List.of(insumoDTO));
91
92          mockMvc.perform(get(BASE_URL + "/categoria/{
    categoriaId}", 10L).contextPath(CONTEXT_PATH))
93                  .andExpect(status().isOk())
94                  .andExpect(jsonPath("$", hasSize(1
    )))
95                  .andExpect(jsonPath("$[0].categoria"
    ).value(CATEGORIA));
96      }
97
```

```
 98     @Test
 99     void obtenerPorId_ok() throws Exception {
100         when(insumoService.obtenerPorId(ID)).
    thenReturn(insumoDTO);
101 
102         mockMvc.perform(get(BASE_URL + "/{id}", ID).
    contextPath(CONTEXT_PATH))
103                 .andExpect(status().isOk())
104                 .andExpect(jsonPath("$.id").value(ID
    ))
105                 .andExpect(jsonPath("$.nombre").
    value(NOMBRE));
106     }
107 
108     @Test
109     void obtenerPorId_notFound() throws Exception {
110         when(insumoService.obtenerPorId(99L))
111                 .thenThrow(new
    EntityNotFoundException("No se encontro Insumo con
    id 99"));
112 
113         mockMvc.perform(get(BASE_URL + "/{id}", 99L
    ).contextPath(CONTEXT_PATH))
114                 .andExpect(status().isNotFound())
115                 .andExpect(jsonPath("$.status").
    value(404));
116     }
117 
118     @Test
119     void crear_ok() throws Exception {
120         InsumoRequestDTO request = new
    InsumoRequestDTO(NOMBRE, 10L, 20, true);
121         when(insumoService.crearInsumo(any(
    InsumoRequestDTO.class))).thenReturn(insumoDTO);
122 
123         mockMvc.perform(post(BASE_URL)
124                         .contextPath(CONTEXT_PATH)
125                         .contentType(MediaType.
    APPLICATION_JSON)
126                         .content(objectMapper.
    writeValueAsString(request)))
```

```
127                 .andExpect(status().isOk())
128                 .andExpect(jsonPath("$.id").value(ID
    ))
129                 .andExpect(jsonPath("$.nombre").
    value(NOMBRE));
130     }
131
132     @Test
133     void crear_nombreDuplicado_badRequest() throws
    Exception {
134         InsumoRequestDTO request = new
    InsumoRequestDTO(NOMBRE, 10L, 20, true);
135         when(insumoService.crearInsumo(any(
    InsumoRequestDTO.class)))
136                 .thenThrow(new
    IllegalArgumentException("Ya existe un insumo con
    ese nombre"));
137
138         mockMvc.perform(post(BASE_URL)
139                         .contextPath(CONTEXT_PATH)
140                         .contentType(MediaType.
    APPLICATION_JSON)
141                         .content(objectMapper.
    writeValueAsString(request)))
142                 .andExpect(status().isBadRequest())
143                 .andExpect(jsonPath("$.status").
    value(400))
144                 .andExpect(jsonPath("$.detalles[0]"
    ).value("Ya existe un insumo con ese nombre"));
145     }
146
147     @Test
148     void actualizar_ok() throws Exception {
149         InsumoRequestDTO request = new
    InsumoRequestDTO("Adrenalina 2mg", 10L, 25, true);
150         InsumoDTO updatedDTO = InsumoDTO.builder()
151                 .id(ID)
152                 .nombre("Adrenalina 2mg")
153                 .categoria(CATEGORIA)
154                 .puntoControl(25)
155                 .esCritico(true)
```

```
156                 .build();
157         when(insumoService.actualizarInsumo(eq(ID),
    any(InsumoRequestDTO.class))).thenReturn(updatedDTO
    );
158 
159         mockMvc.perform(put(BASE_URL + "/{id}", ID)
160                         .contextPath(CONTEXT_PATH)
161                         .contentType(MediaType.
    APPLICATION_JSON)
162                         .content(objectMapper.
    writeValueAsString(request)))
163                 .andExpect(status().isOk())
164                 .andExpect(jsonPath("$.id").value(ID
    ))
165                 .andExpect(jsonPath("$.nombre").
    value("Adrenalina 2mg"));
166     }
167 
168     @Test
169     void actualizar_notFound() throws Exception {
170         InsumoRequestDTO request = new
    InsumoRequestDTO("Insumo", 10L, 5, false);
171         when(insumoService.actualizarInsumo(eq(99L
    ), any(InsumoRequestDTO.class)))
172                 .thenThrow(new
    EntityNotFoundException("No se encontro Insumo con
    id 99"));
173 
174         mockMvc.perform(put(BASE_URL + "/{id}", 99L)
175                         .contextPath(CONTEXT_PATH)
176                         .contentType(MediaType.
    APPLICATION_JSON)
177                         .content(objectMapper.
    writeValueAsString(request)))
178                 .andExpect(status().isNotFound())
179                 .andExpect(jsonPath("$.status").
    value(404));
180     }
181 
182     @Test
183     void actualizar_nombreDuplicado_badRequest()
```

```
183 throws Exception {
184         InsumoRequestDTO request = new
    InsumoRequestDTO(NOMBRE, 10L, 20, true);
185         when(insumoService.actualizarInsumo(eq(ID),
    any(InsumoRequestDTO.class)))
186                 .thenThrow(new
    IllegalArgumentException("Ya existe un insumo con
    ese nombre"));
187 
188         mockMvc.perform(put(BASE_URL + "/{id}", ID)
189                         .contextPath(CONTEXT_PATH)
190                         .contentType(MediaType.
    APPLICATION_JSON)
191                         .content(objectMapper.
    writeValueAsString(request)))
192                 .andExpect(status().isBadRequest())
193                 .andExpect(jsonPath("$.status").
    value(400))
194                 .andExpect(jsonPath("$.detalles[0]"
    ).value("Ya existe un insumo con ese nombre"));
195     }
196 
197     @Test
198     void eliminar_ok() throws Exception {
199         doNothing().when(insumoService).
    eliminarPorId(ID);
200 
201         mockMvc.perform(delete(BASE_URL + "/{id}",
    ID).contextPath(CONTEXT_PATH))
202                 .andExpect(status().isNoContent());
203     }
204 
205     @Test
206     void eliminar_notFound() throws Exception {
207         doThrow(new EntityNotFoundException("No se
    encontro Insumo con id 99"))
208                 .when(insumoService).eliminarPorId(
    99L);
209 
210         mockMvc.perform(delete(BASE_URL + "/{id}",
    99L).contextPath(CONTEXT_PATH))
```

```
211                 .andExpect(status().isNotFound())
212                 .andExpect(jsonPath("$.status").
    value(404));
213     }
214 }
215
```

```java
package com.tup.tfi.control107.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.tup.tfi.control107.config.SecurityConfig;
import com.tup.tfi.control107.dto.UsuarioDTO;
import com.tup.tfi.control107.dto.request.
UsuarioRequestDTO;
import com.tup.tfi.control107.model.enums.RolUsuario;
import com.tup.tfi.control107.service.UsuarioService;
import com.tup.tfi.control107.util.JwtUtil;
import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.
Autowired;
import org.springframework.boot.webmvc.test.
autoconfigure.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.security.test.context.
support.WithMockUser;
import org.springframework.test.context.bean.override
.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.List;

import static org.hamcrest.Matchers.hasSize;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.doNothing;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.
request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.
result.MockMvcResultMatchers.*;

@WebMvcTest(UsuarioController.class)
@Import(SecurityConfig.class)
@WithMockUser(roles = "ADMINISTRADOR")
```

```
35 class UsuarioControllerTest {
36
37
38     @Autowired
39     private MockMvc mockMvc;
40
41     private final ObjectMapper objectMapper = new
   ObjectMapper();
42
43     @MockitoBean
44     private UsuarioService usuarioService;
45
46     @MockitoBean
47     private JwtUtil jwtUtil;
48
49     private UsuarioDTO usuarioDTO;
50     private static final Long ID = 1L;
51     private static final String NOMBRE = "Hugo
   Catalan";
52     private static final String DNI = "30111222";
53     private static final String EMAIL = "hugo.catalan
   @control107.gob.ar";
54     private static final String CONTEXT_PATH = "/api"
   ;
55     private static final String BASE_URL = "/api/v1/
   usuarios";
56
57     @BeforeEach
58     void setUp() {
59         usuarioDTO = UsuarioDTO.builder()
60                 .id(ID)
61                 .nombre(NOMBRE)
62                 .dni(DNI)
63                 .email(EMAIL)
64                 .rol(RolUsuario.ADMINISTRADOR)
65                 .activo(true)
66                 .build();
67     }
68
69     @Test
70     void listarTodos_ok() throws Exception {
```

```
71          when(usuarioService.listarTodos()).
    thenReturn(List.of(usuarioDTO));
72
73          mockMvc.perform(get(BASE_URL).contextPath(
    CONTEXT_PATH))
74                  .andExpect(status().isOk())
75                  .andExpect(jsonPath("$", hasSize(1
    )))
76                  .andExpect(jsonPath("$[0].id").value
    (ID))
77                  .andExpect(jsonPath("$[0].nombre").
    value(NOMBRE))
78                  .andExpect(jsonPath("$[0].dni").
    value(DNI))
79                  .andExpect(jsonPath("$[0].email").
    value(EMAIL));
80      }
81
82      @Test
83      void obtenerPorId_ok() throws Exception {
84          when(usuarioService.obtenerPorId(ID)).
    thenReturn(usuarioDTO);
85
86          mockMvc.perform(get(BASE_URL + "/{id}", ID).
    contextPath(CONTEXT_PATH))
87                  .andExpect(status().isOk())
88                  .andExpect(jsonPath("$.id").value(ID
    ))
89                  .andExpect(jsonPath("$.nombre").
    value(NOMBRE))
90                  .andExpect(jsonPath("$.dni").value(
    DNI));
91      }
92
93      @Test
94      void obtenerPorId_notFound() throws Exception {
95          when(usuarioService.obtenerPorId(99L))
96                  .thenThrow(new
    EntityNotFoundException("No se encontro Usuario con
    id 99"));
97
```

```
 98          mockMvc.perform(get(BASE_URL + "/{id}", 99L
    ).contextPath(CONTEXT_PATH))
 99                  .andExpect(status().isNotFound())
100                  .andExpect(jsonPath("$.status").
    value(404));
101      }
102
103      @Test
104      void crear_ok() throws Exception {
105          UsuarioRequestDTO request = new
    UsuarioRequestDTO(NOMBRE, DNI, EMAIL, RolUsuario.
    ADMINISTRADOR, "1234", true);
106          when(usuarioService.crearUsuario(any(
    UsuarioRequestDTO.class))).thenReturn(usuarioDTO);
107
108          mockMvc.perform(post(BASE_URL)
109                          .contextPath(CONTEXT_PATH)
110                          .contentType(MediaType.
    APPLICATION_JSON)
111                          .content(objectMapper.
    writeValueAsString(request)))
112                  .andExpect(status().isOk())
113                  .andExpect(jsonPath("$.id").value(ID
    ))
114                  .andExpect(jsonPath("$.nombre").
    value(NOMBRE))
115                  .andExpect(jsonPath("$.dni").value(
    DNI));
116      }
117
118      @Test
119      void crear_dniDuplicado_badRequest() throws
    Exception {
120          UsuarioRequestDTO request = new
    UsuarioRequestDTO(NOMBRE, DNI, EMAIL, RolUsuario.
    ADMINISTRADOR, "1234", true);
121          when(usuarioService.crearUsuario(any(
    UsuarioRequestDTO.class)))
122                  .thenThrow(new
    IllegalArgumentException("Ya existe un usuario con
    ese DNI"));
```

```
123
124         mockMvc.perform(post(BASE_URL)
125                         .contextPath(CONTEXT_PATH)
126                         .contentType(MediaType.
    APPLICATION_JSON)
127                         .content(objectMapper.
    writeValueAsString(request)))
128                 .andExpect(status().isBadRequest())
129                 .andExpect(jsonPath("$.status").
    value(400))
130                 .andExpect(jsonPath("$.detalles[0]"
    ).value("Ya existe un usuario con ese DNI"));
131     }
132
133     @Test
134     void crear_emailDuplicado_badRequest() throws
    Exception {
135         UsuarioRequestDTO request = new
    UsuarioRequestDTO(NOMBRE, DNI, EMAIL, RolUsuario.
    ADMINISTRADOR, "1234", true);
136         when(usuarioService.crearUsuario(any(
    UsuarioRequestDTO.class)))
137                 .thenThrow(new
    IllegalArgumentException("Ya existe un usuario con
    ese email"));
138
139         mockMvc.perform(post(BASE_URL)
140                         .contextPath(CONTEXT_PATH)
141                         .contentType(MediaType.
    APPLICATION_JSON)
142                         .content(objectMapper.
    writeValueAsString(request)))
143                 .andExpect(status().isBadRequest())
144                 .andExpect(jsonPath("$.status").
    value(400))
145                 .andExpect(jsonPath("$.detalles[0]"
    ).value("Ya existe un usuario con ese email"));
146     }
147
148     @Test
149     void actualizar_ok() throws Exception {
```

```
150         UsuarioRequestDTO request = new
    UsuarioRequestDTO("Hugo C.", DNI, EMAIL, RolUsuario.
    ADMINISTRADOR, "1234", true);
151         UsuarioDTO updated = UsuarioDTO.builder()
152                 .id(ID)
153                 .nombre("Hugo C.")
154                 .dni(DNI)
155                 .email(EMAIL)
156                 .rol(RolUsuario.ADMINISTRADOR)
157                 .activo(true)
158                 .build();
159         when(usuarioService.actualizarUsuario(eq(ID
    ), any(UsuarioRequestDTO.class))).thenReturn(updated
    );
160
161         mockMvc.perform(put(BASE_URL + "/{id}", ID)
162                         .contextPath(CONTEXT_PATH)
163                         .contentType(MediaType.
    APPLICATION_JSON)
164                         .content(objectMapper.
    writeValueAsString(request)))
165                 .andExpect(status().isOk())
166                 .andExpect(jsonPath("$.id").value(ID
    ))
167                 .andExpect(jsonPath("$.nombre").
    value("Hugo C."));
168     }
169
170     @Test
171     void actualizar_notFound() throws Exception {
172         UsuarioRequestDTO request = new
    UsuarioRequestDTO(NOMBRE, DNI, EMAIL, RolUsuario.
    ADMINISTRADOR, "1234", true);
173         when(usuarioService.actualizarUsuario(eq(99L
    ), any(UsuarioRequestDTO.class)))
174                 .thenThrow(new
    EntityNotFoundException("No se encontro Usuario con
    id 99"));
175
176         mockMvc.perform(put(BASE_URL + "/{id}", 99L)
177                         .contextPath(CONTEXT_PATH)
```

```
178                         .contentType(MediaType.
    APPLICATION_JSON)
179                         .content(objectMapper.
    writeValueAsString(request)))
180                 .andExpect(status().isNotFound())
181                 .andExpect(jsonPath("$.status").
    value(404));
182     }
183 
184     @Test
185     void actualizar_dniDuplicado_badRequest() throws
     Exception {
186         UsuarioRequestDTO request = new
    UsuarioRequestDTO(NOMBRE, DNI, EMAIL, RolUsuario.
    ADMINISTRADOR, "1234", true);
187         when(usuarioService.actualizarUsuario(eq(ID
    ), any(UsuarioRequestDTO.class)))
188                 .thenThrow(new
    IllegalArgumentException("Ya existe un usuario con
    ese DNI"));
189 
190         mockMvc.perform(put(BASE_URL + "/{id}", ID)
191                         .contextPath(CONTEXT_PATH)
192                         .contentType(MediaType.
    APPLICATION_JSON)
193                         .content(objectMapper.
    writeValueAsString(request)))
194                 .andExpect(status().isBadRequest())
195                 .andExpect(jsonPath("$.status").
    value(400));
196     }
197 
198     @Test
199     void actualizar_emailDuplicado_badRequest()
    throws Exception {
200         UsuarioRequestDTO request = new
    UsuarioRequestDTO(NOMBRE, DNI, EMAIL, RolUsuario.
    ADMINISTRADOR, "1234", true);
201         when(usuarioService.actualizarUsuario(eq(ID
    ), any(UsuarioRequestDTO.class)))
202                 .thenThrow(new
```

```
202 IllegalArgumentException("Ya existe un usuario con
    ese email"));
203
204         mockMvc.perform(put(BASE_URL + "/{id}", ID)
205                         .contextPath(CONTEXT_PATH)
206                         .contentType(MediaType.
    APPLICATION_JSON)
207                         .content(objectMapper.
    writeValueAsString(request)))
208                 .andExpect(status().isBadRequest())
209                 .andExpect(jsonPath("$.status").
    value(400));
210     }
211
212     @Test
213     void eliminar_ok() throws Exception {
214         doNothing().when(usuarioService).
    eliminarPorId(ID);
215
216         mockMvc.perform(delete(BASE_URL + "/{id}",
    ID).contextPath(CONTEXT_PATH))
217                 .andExpect(status().isNoContent());
218     }
219
220     @Test
221     void eliminar_notFound() throws Exception {
222         doThrow(new EntityNotFoundException("No se
    encontro Usuario con id 99"))
223                 .when(usuarioService).eliminarPorId(
    99L);
224
225         mockMvc.perform(delete(BASE_URL + "/{id}",
    99L).contextPath(CONTEXT_PATH))
226                 .andExpect(status().isNotFound())
227                 .andExpect(jsonPath("$.status").
    value(404));
228     }
229 }
230
```

```java
package com.tup.tfi.control107.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.tup.tfi.control107.config.SecurityConfig;
import com.tup.tfi.control107.dto.AmbulanciaDTO;
import com.tup.tfi.control107.dto.request.
  AmbulanciaRequestDTO;
import com.tup.tfi.control107.service.
  AmbulanciaService;
import com.tup.tfi.control107.util.JwtUtil;
import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.
  Autowired;
import org.springframework.boot.webmvc.test.
  autoconfigure.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.security.test.context.
  support.WithMockUser;
import org.springframework.test.context.bean.override
  .mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.List;
import java.util.Set;

import static org.hamcrest.Matchers.hasSize;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.doNothing;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.
  request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.
  result.MockMvcResultMatchers.*;

@WebMvcTest(AmbulanciaController.class)
@Import(SecurityConfig.class)
```

```
34 @WithMockUser(roles = "ADMINISTRADOR")
35 class AmbulanciaControllerTest {
36
37
38     @Autowired
39     private MockMvc mockMvc;
40
41     private final ObjectMapper objectMapper = new
   ObjectMapper();
42
43     @MockitoBean
44     private AmbulanciaService ambulanciaService;
45
46     @MockitoBean
47     private JwtUtil jwtUtil;
48
49     private AmbulanciaDTO ambulanciaDTO;
50     private static final Long ID = 1L;
51     private static final String PATENTE = "AB123CD";
52     private static final String CONTEXT_PATH = "/api"
   ;
53     private static final String BASE_URL = "/api/v1/
   ambulancias";
54
55     @BeforeEach
56     void setUp() {
57         ambulanciaDTO = AmbulanciaDTO.builder()
58                 .id(ID)
59                 .patente(PATENTE)
60                 .disponible(true)
61                 .insumos(Set.of())
62                 .build();
63     }
64
65     @Test
66     void listarTodas_ok() throws Exception {
67         when(ambulanciaService.listarTodas()).
   thenReturn(List.of(ambulanciaDTO));
68
69         mockMvc.perform(get(BASE_URL).contextPath(
   CONTEXT_PATH))
```

```
70                  .andExpect(status().isOk())
71                  .andExpect(jsonPath("$", hasSize(1
   )))
72                  .andExpect(jsonPath("$[0].id").value
   (ID))
73                  .andExpect(jsonPath("$[0].patente").
   value(PATENTE))
74                  .andExpect(jsonPath("$[0].disponible
   ").value(true));
75      }
76
77      @Test
78      void obtenerPorId_ok() throws Exception {
79          when(ambulanciaService.obtenerPorId(ID)).
   thenReturn(ambulanciaDTO);
80
81          mockMvc.perform(get(BASE_URL + "/{id}", ID).
   contextPath(CONTEXT_PATH))
82                  .andExpect(status().isOk())
83                  .andExpect(jsonPath("$.id").value(ID
   ))
84                  .andExpect(jsonPath("$.patente").
   value(PATENTE));
85      }
86
87      @Test
88      void obtenerPorId_notFound() throws Exception {
89          when(ambulanciaService.obtenerPorId(99L))
90                  .thenThrow(new
   EntityNotFoundException("No se encontro Ambulancia
   con id 99"));
91
92          mockMvc.perform(get(BASE_URL + "/{id}", 99L
   ).contextPath(CONTEXT_PATH))
93                  .andExpect(status().isNotFound())
94                  .andExpect(jsonPath("$.status").
   value(404));
95      }
96
97      @Test
98      void guardar_ok() throws Exception {
```

```
 99          AmbulanciaRequestDTO request = new
    AmbulanciaRequestDTO(PATENTE, true, Set.of());
100          when(ambulanciaService.crearAmbulancia(any(
    AmbulanciaRequestDTO.class))).thenReturn(
    ambulanciaDTO);
101
102          mockMvc.perform(post(BASE_URL)
103                       .contextPath(CONTEXT_PATH)
104                       .contentType(MediaType.
    APPLICATION_JSON)
105                       .content(objectMapper.
    writeValueAsString(request)))
106                  .andExpect(status().isOk())
107                  .andExpect(jsonPath("$.id").value(ID
    ))
108                  .andExpect(jsonPath("$.patente").
    value(PATENTE));
109      }
110
111      @Test
112      void guardar_patenteDuplicada_badRequest()
    throws Exception {
113          AmbulanciaRequestDTO request = new
    AmbulanciaRequestDTO(PATENTE, true, Set.of());
114          when(ambulanciaService.crearAmbulancia(any(
    AmbulanciaRequestDTO.class)))
115                  .thenThrow(new
    IllegalArgumentException("Ya existe una ambulancia
    con esa patente"));
116
117          mockMvc.perform(post(BASE_URL)
118                       .contextPath(CONTEXT_PATH)
119                       .contentType(MediaType.
    APPLICATION_JSON)
120                       .content(objectMapper.
    writeValueAsString(request)))
121                  .andExpect(status().isBadRequest())
122                  .andExpect(jsonPath("$.status").
    value(400))
123                  .andExpect(jsonPath("$.detalles[0]"
    ).value("Ya existe una ambulancia con esa patente"
```

```
123 ));
124     }
125 
126     @Test
127     void actualizar_ok() throws Exception {
128         AmbulanciaRequestDTO request = new
    AmbulanciaRequestDTO("AB 999 CD", true, Set.of());
129         AmbulanciaDTO updated = AmbulanciaDTO.
    builder()
130                 .id(ID)
131                 .patente("AB 999 CD")
132                 .disponible(true)
133                 .insumos(Set.of())
134                 .build();
135         when(ambulanciaService.actualizarAmbulancia(
    eq(ID), any(AmbulanciaRequestDTO.class))).thenReturn
    (updated);
136 
137         mockMvc.perform(put(BASE_URL + "/{id}", ID)
138                         .contextPath(CONTEXT_PATH)
139                         .contentType(MediaType.
    APPLICATION_JSON)
140                         .content(objectMapper.
    writeValueAsString(request)))
141                 .andExpect(status().isOk())
142                 .andExpect(jsonPath("$.id").value(ID
    ))
143                 .andExpect(jsonPath("$.patente").
    value("AB 999 CD"));
144     }
145 
146     @Test
147     void actualizar_notFound() throws Exception {
148         AmbulanciaRequestDTO request = new
    AmbulanciaRequestDTO(PATENTE, true, Set.of());
149         when(ambulanciaService.actualizarAmbulancia(
    eq(99L), any(AmbulanciaRequestDTO.class)))
150                 .thenThrow(new
    EntityNotFoundException("No se encontro Ambulancia
    con id 99"));
151 
```

```
152         mockMvc.perform(put(BASE_URL + "/{id}", 99L)
153                         .contextPath(CONTEXT_PATH)
154                         .contentType(MediaType.
    APPLICATION_JSON)
155                         .content(objectMapper.
    writeValueAsString(request)))
156                 .andExpect(status().isNotFound())
157                 .andExpect(jsonPath("$.status").
    value(404));
158     }
159 
160     @Test
161     void actualizar_patenteDuplicada_badRequest()
    throws Exception {
162         AmbulanciaRequestDTO request = new
    AmbulanciaRequestDTO(PATENTE, true, Set.of());
163         when(ambulanciaService.actualizarAmbulancia(
    eq(ID), any(AmbulanciaRequestDTO.class)))
164                 .thenThrow(new
    IllegalArgumentException("Ya existe una ambulancia
    con esa patente"));
165 
166         mockMvc.perform(put(BASE_URL + "/{id}", ID)
167                         .contextPath(CONTEXT_PATH)
168                         .contentType(MediaType.
    APPLICATION_JSON)
169                         .content(objectMapper.
    writeValueAsString(request)))
170                 .andExpect(status().isBadRequest())
171                 .andExpect(jsonPath("$.status").
    value(400));
172     }
173 
174     @Test
175     void eliminar_ok() throws Exception {
176         doNothing().when(ambulanciaService).
    eliminarPorId(ID);
177 
178         mockMvc.perform(delete(BASE_URL + "/{id}",
    ID).contextPath(CONTEXT_PATH))
179                 .andExpect(status().isNoContent());
```

```
180     }
181
182     @Test
183     void eliminar_notFound() throws Exception {
184         doThrow(new EntityNotFoundException("No se
    encontro Ambulancia con id 99"))
185                 .when(ambulanciaService).
    eliminarPorId(99L);
186
187         mockMvc.perform(delete(BASE_URL + "/{id}",
    99L).contextPath(CONTEXT_PATH))
188                 .andExpect(status().isNotFound())
189                 .andExpect(jsonPath("$.status").
    value(404));
190     }
191 }
192
```

```java
package com.tup.tfi.control107.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.tup.tfi.control107.dto.CategoriaInsumoDTO;
import com.tup.tfi.control107.dto.request.
  CategoriaInsumoRequestDTO;
import com.tup.tfi.control107.service.
  CategoriaInsumoService;
import com.tup.tfi.control107.util.JwtUtil;
import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.
  Autowired;
import org.springframework.boot.webmvc.test.
  autoconfigure.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.security.test.context.
  support.WithMockUser;
import org.springframework.test.context.bean.override
  .mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.List;

import static org.hamcrest.Matchers.hasSize;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.doNothing;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.
  request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.
  result.MockMvcResultMatchers.*;

import org.springframework.context.annotation.Import;
import com.tup.tfi.control107.config.SecurityConfig;

@WebMvcTest(CategoriaInsumoController.class)
@Import(SecurityConfig.class)
```

```
34 @WithMockUser(roles = "ADMINISTRADOR")
35 class CategoriaInsumoControllerTest {
36
37
38     @Autowired
39     private MockMvc mockMvc;
40
41     private final ObjectMapper objectMapper = new
   ObjectMapper();
42
43     @MockitoBean
44     private CategoriaInsumoService
   categoriaInsumoService;
45
46     // Resuelve UnsatisfiedDependencyException en
   JwtFilter
47     @MockitoBean
48     private JwtUtil jwtUtil;
49
50     private CategoriaInsumoDTO categoriaInsumoDTO;
51     private List<CategoriaInsumoDTO>
   categoriaInsumoDTOList;
52
53     private static final long ID = 1L;
54     private static final String NOMBRE = "test-
   categoria";
55     private static final String COLOR = "#000000";
56     private static final String CONTEXT_PATH = "/api"
   ;
57     private static final String BASE_URL = "/api/v1/
   categorias";
58
59     @BeforeEach
60     void setUp() {
61         categoriaInsumoDTO = new CategoriaInsumoDTO
   ();
62         categoriaInsumoDTO.setId(ID);
63         categoriaInsumoDTO.setNombre(NOMBRE);
64         categoriaInsumoDTO.setColor(COLOR);
65         categoriaInsumoDTOList = List.of(
   categoriaInsumoDTO);
```

```
66      }
67
68      @Test
69      void listarTodas() throws Exception {
70          // Arrange
71          when(categoriaInsumoService.listarTodas()).
    thenReturn(categoriaInsumoDTOList);
72
73          // Act & Assert
74          mockMvc.perform(get(BASE_URL).contextPath(
    CONTEXT_PATH))
75                  .andExpect(status().isOk())
76                  .andExpect(jsonPath("$", hasSize(1
    )))
77                  .andExpect(jsonPath("$[0].id").value
    (ID))
78                  .andExpect(jsonPath("$[0].nombre").
    value(NOMBRE))
79                  .andExpect(jsonPath("$[0].color").
    value(COLOR));
80      }
81
82      @Test
83      void obtenerPorId_ok() throws Exception {
84          // Arrange
85          when(categoriaInsumoService.obtenerPorId(ID
    )).thenReturn(categoriaInsumoDTO);
86
87          // Act & Assert
88          mockMvc.perform(get(BASE_URL + "/{id}", ID).
    contextPath(CONTEXT_PATH))
89                  .andExpect(status().isOk())
90                  .andExpect(jsonPath("$.id").value(ID
    ))
91                  .andExpect(jsonPath("$.nombre").
    value(NOMBRE))
92                  .andExpect(jsonPath("$.color").value
    (COLOR));
93      }
94
95      @Test
```

```
 96      void obtenerPorId_notFound() throws Exception {
 97          // Arrange
 98          when(categoriaInsumoService.obtenerPorId(99L
     ))
 99                  .thenThrow(new
     EntityNotFoundException("No se encontro
     CategoriaInsumo con id 99"));
100
101          // Act & Assert
102          mockMvc.perform(get(BASE_URL + "/{id}", 99L
     ).contextPath(CONTEXT_PATH))
103                  .andExpect(status().isNotFound())
104                  .andExpect(jsonPath("$.status").
     value(404))
105                  .andExpect(jsonPath("$.error").value
     ("Recurso no encontrado"));
106      }
107
108      @Test
109      void crear_ok() throws Exception {
110          // Arrange
111          CategoriaInsumoRequestDTO request = new
     CategoriaInsumoRequestDTO(NOMBRE, COLOR);
112          when(categoriaInsumoService.crearCategoria(
     any(CategoriaInsumoRequestDTO.class)))
113                  .thenReturn(categoriaInsumoDTO);
114
115          // Act & Assert
116          mockMvc.perform(post(BASE_URL)
117                          .contextPath(CONTEXT_PATH)
118                          .contentType(MediaType.
     APPLICATION_JSON)
119                          .content(objectMapper.
     writeValueAsString(request)))
120                  .andExpect(status().isOk())
121                  .andExpect(jsonPath("$.id").value(ID
     ))
122                  .andExpect(jsonPath("$.nombre").
     value(NOMBRE))
123                  .andExpect(jsonPath("$.color").value
     (COLOR));
```

```
124     }
125
126     @Test
127     void crear_nombreDuplicado_badRequest() throws
    Exception {
128         // Arrange
129         CategoriaInsumoRequestDTO request = new
    CategoriaInsumoRequestDTO(NOMBRE, COLOR);
130         when(categoriaInsumoService.crearCategoria(
    any(CategoriaInsumoRequestDTO.class)))
131                 .thenThrow(new
    IllegalArgumentException("Ya existe una categoría
    con ese nombre"));
132
133         // Act & Assert
134         mockMvc.perform(post(BASE_URL)
135                         .contextPath(CONTEXT_PATH)
136                         .contentType(MediaType.
    APPLICATION_JSON)
137                         .content(objectMapper.
    writeValueAsString(request)))
138                 .andExpect(status().isBadRequest())
139                 .andExpect(jsonPath("$.status").
    value(400))
140                 .andExpect(jsonPath("$.error").value
    ("Error de validación"))
141                 .andExpect(jsonPath("$.detalles[0]"
    ).value("Ya existe una categoría con ese nombre"));
142     }
143
144     @Test
145     void actualizar_ok() throws Exception {
146         // Arrange
147         CategoriaInsumoRequestDTO request = new
    CategoriaInsumoRequestDTO("categoria-modificada", "#
    FFFFFF");
148         CategoriaInsumoDTO updatedDTO = new
    CategoriaInsumoDTO();
149         updatedDTO.setId(ID);
150         updatedDTO.setNombre("categoria-modificada"
    );
```

```
151         updatedDTO.setColor("#FFFFFF");
152 
153         when(categoriaInsumoService.
    actualizarCategoria(eq(ID), any(
    CategoriaInsumoRequestDTO.class)))
154                 .thenReturn(updatedDTO);
155 
156         // Act & Assert
157         mockMvc.perform(put(BASE_URL + "/{id}", ID)
158                         .contextPath(CONTEXT_PATH)
159                         .contentType(MediaType.
    APPLICATION_JSON)
160                         .content(objectMapper.
    writeValueAsString(request)))
161                 .andExpect(status().isOk())
162                 .andExpect(jsonPath("$.id").value(ID
    ))
163                 .andExpect(jsonPath("$.nombre").
    value("categoria-modificada"))
164                 .andExpect(jsonPath("$.color").value
    ("#FFFFFF"));
165     }
166 
167     @Test
168     void actualizar_notFound() throws Exception {
169         // Arrange
170         CategoriaInsumoRequestDTO request = new
    CategoriaInsumoRequestDTO("modificada", "#FFFFFF");
171         when(categoriaInsumoService.
    actualizarCategoria(eq(99L), any(
    CategoriaInsumoRequestDTO.class)))
172                 .thenThrow(new
    EntityNotFoundException("No se encontro
    CategoriaInsumo con id 99"));
173 
174         // Act & Assert
175         mockMvc.perform(put(BASE_URL + "/{id}", 99L)
176                         .contextPath(CONTEXT_PATH)
177                         .contentType(MediaType.
    APPLICATION_JSON)
178                         .content(objectMapper.
```

```
178 writeValueAsString(request)))
179                 .andExpect(status().isNotFound())
180                 .andExpect(jsonPath("$.status").
    value(404));
181     }
182 
183     @Test
184     void actualizar_nombreDuplicado_badRequest()
    throws Exception {
185         // Arrange
186         CategoriaInsumoRequestDTO request = new
    CategoriaInsumoRequestDTO(NOMBRE, COLOR);
187         when(categoriaInsumoService.
    actualizarCategoria(eq(ID), any(
    CategoriaInsumoRequestDTO.class)))
188                 .thenThrow(new
    IllegalArgumentException("Ya existe una categoría
    con ese nombre"));
189 
190         // Act & Assert
191         mockMvc.perform(put(BASE_URL + "/{id}", ID)
192                         .contextPath(CONTEXT_PATH)
193                         .contentType(MediaType.
    APPLICATION_JSON)
194                         .content(objectMapper.
    writeValueAsString(request)))
195                 .andExpect(status().isBadRequest())
196                 .andExpect(jsonPath("$.status").
    value(400))
197                 .andExpect(jsonPath("$.error").value
    ("Error de validación"));
198     }
199 
200     @Test
201     void eliminar_ok() throws Exception {
202         // Arrange
203         doNothing().when(categoriaInsumoService).
    eliminarPorId(ID);
204 
205         // Act & Assert
206         mockMvc.perform(delete(BASE_URL + "/{id}",
```

```
206 ID).contextPath(CONTEXT_PATH))
207                 .andExpect(status().isNoContent());
208     }
209 
210     @Test
211     void eliminar_notFound() throws Exception {
212         // Arrange
213         doThrow(new EntityNotFoundException("No se
    encontro CategoriaInsumo con id 99"))
214                 .when(categoriaInsumoService).
    eliminarPorId(99L);
215 
216         // Act & Assert
217         mockMvc.perform(delete(BASE_URL + "/{id}",
    99L).contextPath(CONTEXT_PATH))
218                 .andExpect(status().isNotFound())
219                 .andExpect(jsonPath("$.status").
    value(404));
220     }
221 }
```

```
# Test Configuration
spring.application.name=control107-test

# H2 Database Configuration for Testing
spring.datasource.url=jdbc:h2:mem:testdb;
DB_CLOSE_DELAY=-1;MODE=MySQL;NON_KEYWORDS=KEY,VALUE
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA Configuration for Testing
spring.jpa.database-platform=org.hibernate.dialect.
H2Dialect
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=false

# Disable H2 Console in tests
spring.h2.console.enabled=false

# Logging Configuration for Tests
logging.level.root=WARN
logging.level.com.pruebatecnica.productcomparison=
DEBUG
```